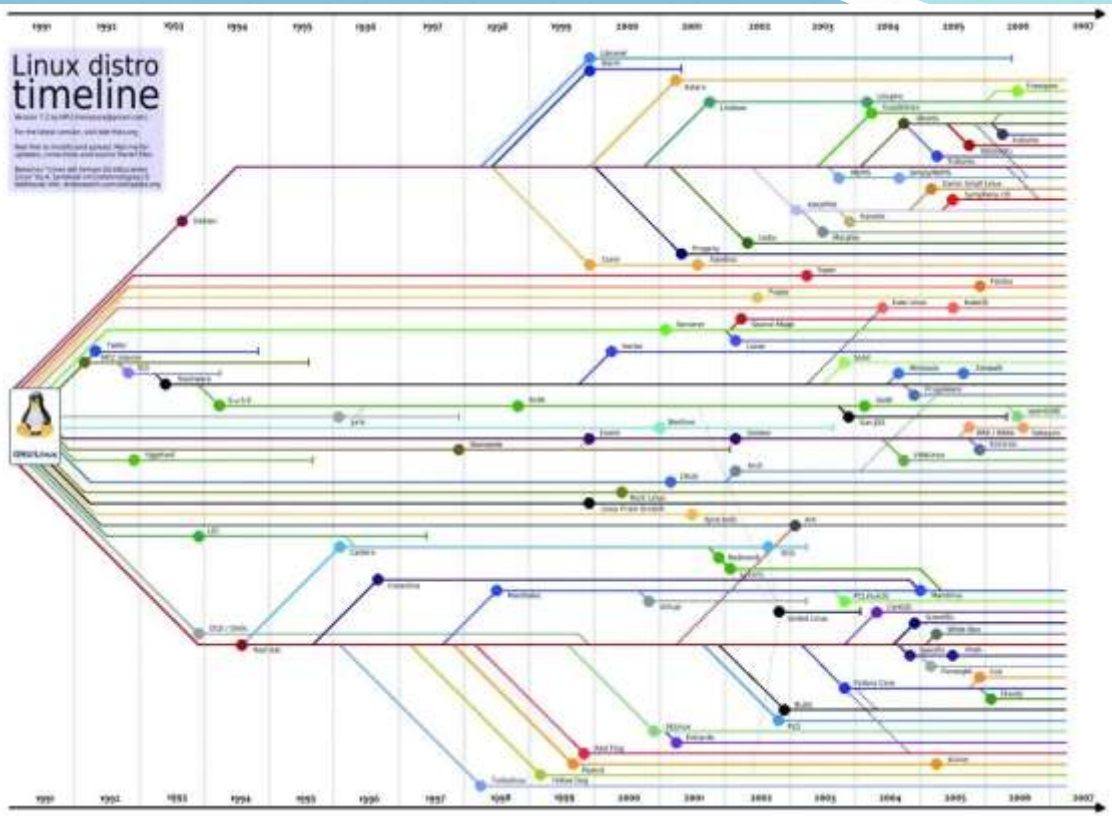
软件工程与方法

第8讲 软件进化

不同的软件发展方式

基于Linux 内核7.2版本的操作系统演化发展模式

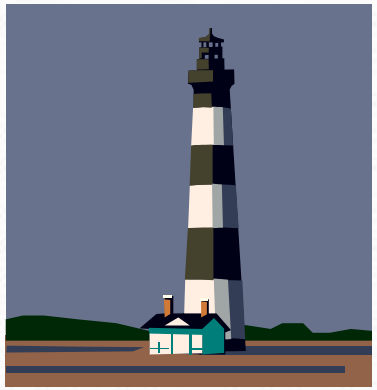


微软操作系统发展历程



主要内容

1. 遗留系统
2. 软件变更
3. 软件维护
4. 软件再工程
5. 配置管理理
6. DevOps



软件进化

1 遗留系统

什么是遗留系统？

- 旧系统又称为遗留系统（legacy system）。
- 遗留系统可能已经运行了10年或20年。
- 遗留系统可能不是当初交付的系统，已经被修改过，可能没有人能完全了解整个系统。
- 抛弃遗留系统或用一个全新的软件系统，风险巨大。



- **更换遗留系统的风险**
  - ① 遗留系统几乎没有完整的描述。
  - ② 业务过程和遗留系统的操作方式紧密“交织”在一起。
  - ③ 重要的业务规则隐藏在软件内部，不可能记录在别的地方。
  - ④ 新系统会遇到难以遇料的问题，开发新软件本身是有风险的。

更换遗留系统的困难

① 系统的不同部分是由不同的团队实现，这样，整个系统的程序设计风格不一致。

② 系统部分或全部是用过时不用的编程语言编写，很难找到会用这种语言的程序员，而且可能需要花高价将系统维护外包。

③ 系统文档通常是不充分和过时的。有时源代码是唯一的系统文档；有时候源代码已经不存在，只剩下系统的可执行版本。

④ 经过多年维护，系统结构可能已经被破坏，理解系统设计的难度逐渐加大。添加的新程序可能以一种特别的接口方式与系统其它部分对接。

⑤ 系统可能对空间利用或运行速度进行了优化，设计的可读性不好。

⑥ 系统所处理的数据保留在一些结构不相容的文件中。这些文件中的数据可能存在重复现象，数据也已经过时、不精确，也不完整。

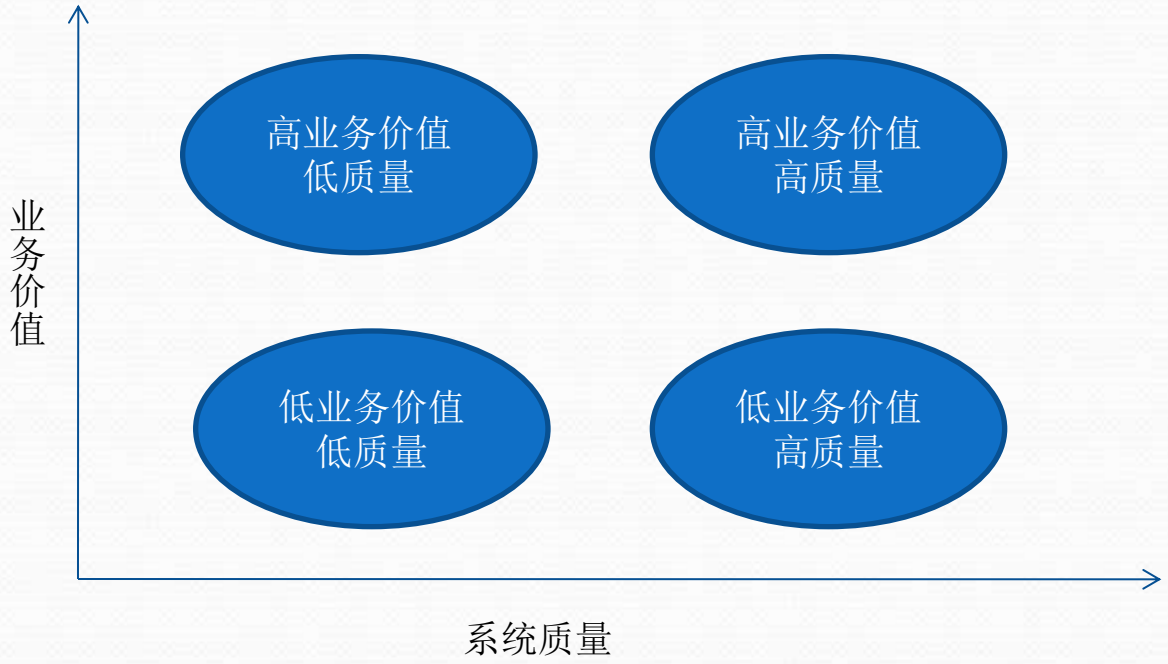
遗留系统评估

- 原有技术不能保证新的质量需求
- 出于战略的考虑
- 原有的技术提供者不再提供支持
- 使用旧有技术的成本太高
- 先对遗留系统做出评估再做如下选择：
  ① 彻底抛弃这个系统
  ② 继续维护这个系统
  ③ 采用某种方式转换系统以改善其可维护性
  ④ 以一个新的系统代替这个系统
- 上述选择并不相互排斥，当一个系统包含多个程序时，可以根据各个部分的实际情况做不同的选择。

评估遗留系统的视角

① 系统质量评估

- 业务过程评估
- 环境评估
- 应用软件评估



② 业务价值评估

- 没有现成可靠的客观方法，对系统的业务价值评估只能是主观的。
- 多视点评估：用户、客户、管理者、IT管理者、资深管理者

环境评估中考虑的因素

因素	问题
供应商的因素	供应商是否存在？供应商经济状况如何，能否继续生存下去？如果供应商已经退出经营，系统是否由其他公司维护？
失败率	硬件是否已经报告有很高的失败率？其支持软件是否会崩溃并被迫重启系统。
年限	硬件和软件的年限有多长？硬件和支持软件越老，就越容易被废弃。虽然其功能正常，但是转移到新系统将相当经济和有效。
性能	系统的性能充分吗？性能问题对系统用户有严重影响吗？
支持的需求	硬件和软件需要哪些局部支持？如果这样的支持成本太高，就要考虑替换系统？
维护成本	硬件维护和软件许可使用费用如何？与新系统相比，越老的硬件维护成本越高，软件则会有较高的年许可使用成本。
互操作性	系统与其它系统接口方面有困难吗？编译器等能在目前的操作系统版本上使用吗？需要硬件仿真吗？

应用评估中的因素

因素	问题
可理解性	对当前系统源代码理解的困难程序如何？其使用的控制结构复杂程度如何？变量有反映其功能的有意义的名字吗？
文档编写	哪些系统文档是有效的？文档是否健全，是否一致，是否反映了当前的状态？
数据	系统有一个清晰的数据模型吗？不同文件中的数据的重复程度如何？系统使用的数据是否是一致的和最新的？
性能	系统的性能充分吗？性能问题对系统用户有严重影响吗？
编程语言	能得到开发系统所用的程序设计语言的最新编译器吗？所用的程序设计语言还用于新系统的开发吗？
配置管理	系统所有版本和所有组成部分都受到配置管理系统的管理吗？对当前系统中的组件的版本都有清晰的描述吗？
测试数据	有无对系统的测试数据？当有新特征加入到系统中时所执行的回归测试有记录吗？
个人技能	员工有能力维护应用吗？是否只有限的几员工了解系统？

可以收集的量化数据

① 系统变更请求的数目
② 系统所用的不同用户界面数目
③ 系统使用的数据量
④ ……



微软VS2015支持古老语言COBOL

COBOL，一种古老的高级编程语言，诞生于1959年5月，比C语言要早十年，目前在财会工作、统计报表及商业数据处理等领域还有在使用。

软件进化

2 软件变更

软件为什么需要变更？

- 无论规模如何，制作一个不需要改变的系统是不可能的。
- 一旦软件进入使用期，新的需求就会时常浮现，已有的需求也会随着软件运行的业务变更而发生改变。
- 软件中的bug需求修改。
- 性能和功能特性也需要改善。
- 软件系统总是在不断进化以响应变更的需求。

微软公司2015年发布的Windows 10安全补丁数目创历史记录，达135个。



软件变更的策略

① 软件维护

- 对软件做局部改变以响应变更的需求，但软件的基本结构保持不变。

② 体系结构的转换

- 这是一种较为彻底的变更方法。
- 通常从集中式、以数据为中心的体系结构向C/S或B/S体系结构转变。

③ 软件再工程

- 不添加新功能。
- 系统经过修改更容易理解和改变。

变更对管理的影响

- 在实施任何变更之初，组织必须沟通：
  - 变更原因
  - 预期效益
  - 将变更纳入工作所需的支持和指导
  - 个人如何受到影响
  - 让每个人参与变更的必要性
- 变更控制
- 变更闭环管理

Lehman定律

定律	描述
连续变更	现实环境中的程序必须进行变更，否则将逐渐失去在相应环境中的作用。
渐增的复杂性	随着程序变更的发生，其结构逐渐变得更加复杂，需要有额外的资源来保持和简化。
大型程序进化	程序进化是一个自我控制过程，系统属性，如规模、两个版本间的时间间隔以及发现错误，对每个版本是不变的。
机构稳定性	在一个程序的生命周期中，其开发速度几乎是常数，并且与所投放到系统中的资源无关。
保持亲密性	在一个系统的生命周期中，每个版本的变更增量都接通近常数。

软件发展的特征：

- Lehman第1定律：
  - 软件产品必须持续改进，否则就会越来越没用。
- Lehman第2定律：
  - 程序的结构可能会随着越来越多的维护的进行而退化。
- Lehman第3定律：
  - 在一个程序的生命周期中，其开发率的数值应当近似为恒定的。

“把今天的方法学应用到昨天的系统以满足明天的需要。”

软件进化

3 软件维护

主要内容

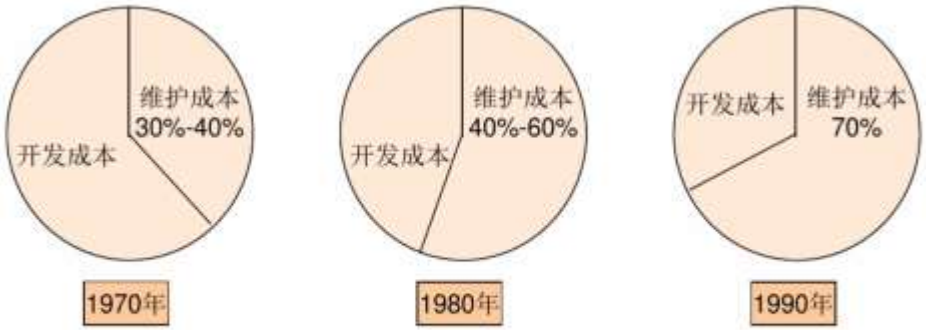
① 软件维护概念
② 软件维护策略
③ 软件修改的副作用
④ 提高软件可维护的方法
⑤ 逆向工程和再工程
⑥ 软件的配置管理



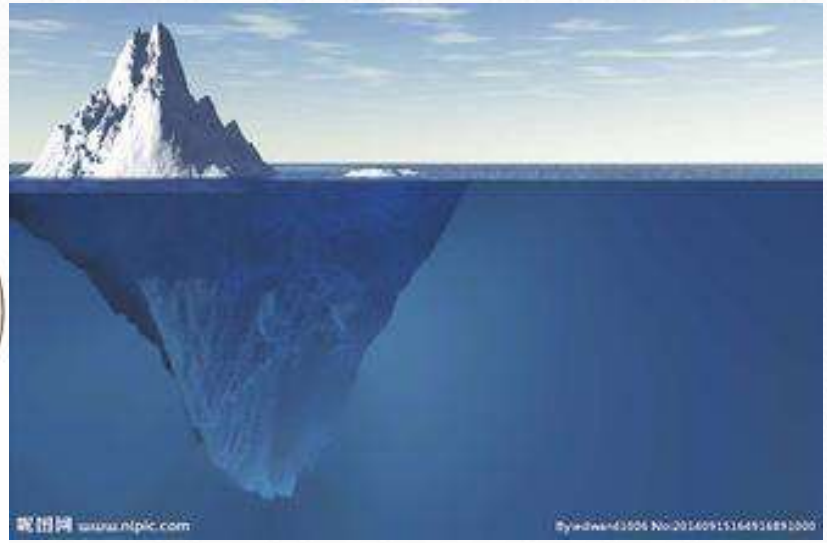
3.1 软件维护的概念

- 在软件运行/维护阶段对软件产品所进行的修改就是维护（maintain）。
- 维护一般不包括重大的修改。
- 软件维护的原因有3种类型：
  - ① 修改Bug
  - ② 增新的功能、改善性能等
  - ③ 为适应环境变化修改软件

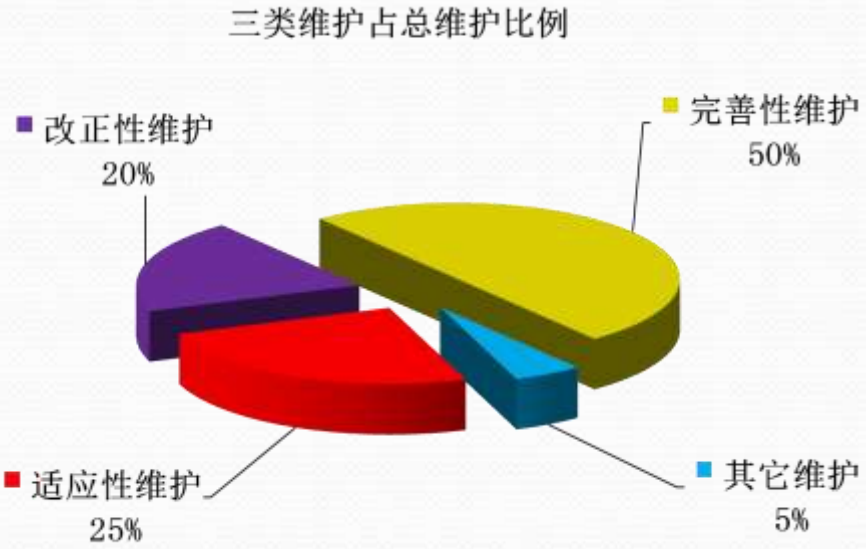
ISO/IEC 14764定义的维护活动：
- 改正性维护（Corrective maintenance）
- 适应性维护（Adaptive maintenance）
- 完善性维护（Perfective maintenance）
- 预防性维护（Preventive maintenance）



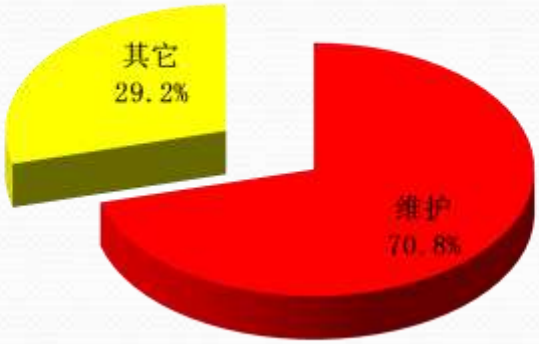
系统维护成本的比例



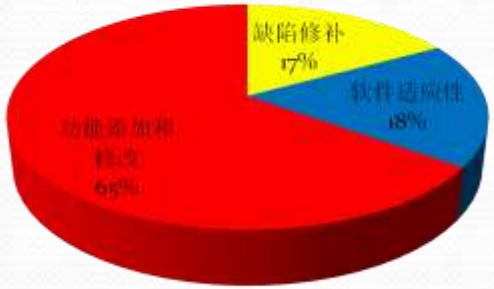
三类维护占总维护的比例



维护在软件生存期所占比例



维护工作量分布



3.2 软件维护的策略

- 使用成熟的技术
- 利用组件技术
- 结构化技术（好程序构架）
- 提高软件的复用性
- 重视文档
- 重视程序风格
- 做好变更控制



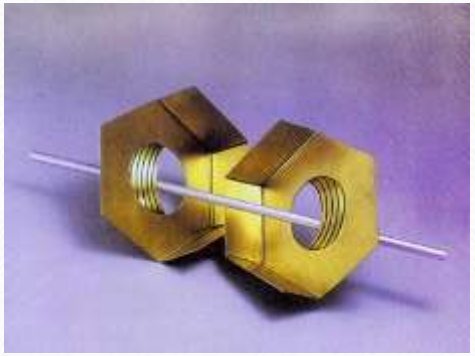
维护工作量模型

$$M = p + Ke^{c-d}$$

- 其中：
  - M：维护中消耗的总工作量；
  - p：生产性工作量；
  - K：一个经验常数；
  - c：因缺乏好的设计和文档面导致复杂性的度量；
  - d：对软件熟悉程序的度量。

3.3 软件可维护性

- 软件维护困难的原因主要是：
  - 文档不完整、不易理解
  - 源程序难以理解、难以修改
- 软件维护的最大挑战是维护过程中可能引入新的错误。



可维护性的度量（1/2）

1) 可理解性
   - 表明人们通过阅读源程序和相关文档，了解程序功能及其如何运行的容易程度。
2) 可靠性
   - 表明一个程序按照用户的要求和设计目标，经给定的一段时间内正确执行的概率。
   - MTTF（Mean Time To Failure），平均无故障时间
   - MTTR（Mean Time To Repair error），平均修复错误时间
3) 可测试性
   - 表明论证程序正确性的容易程度。

可维护性的度量（2/2）

4) 可修改性
   - 表明程序容易修改的程度。
5) 可移植性
   - 表明程序转移到一个新的计算环境的可能性的大小。
6) 效率
   - 表明一程序能执行预定功能而不浪费机器资源的程度。
7) 可用性
   - 一个可用的程序应是易于使用的、能允许用户出错和改变，并尽可能不使用户陷入混乱状态的程序。

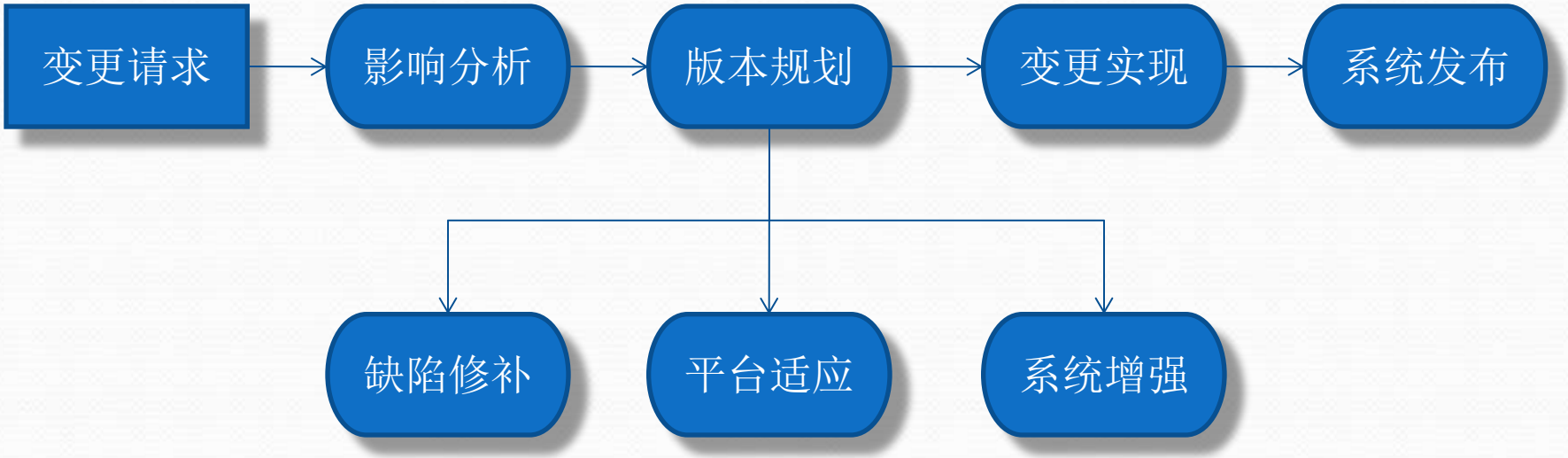
3.4 提高可维护性的方法

- 建立明确的软件质量目标和优先级
  - 某些质量特性是相互促进的，如：可理解性、可测试性和可修改性
  - 另一些质量特性是相互抵触的，如：效率和可移植性、效率和可修改性
  - 可维护性要求每一种质量特性都要满足，但它们的相对重要性因软件的用途及计算环境不同而不同。
- 进行明确的质量保证审查
  - 质量保证审查对于获得和维持软件的质量，是一个很有用的技术。
  - 除了保证软件得到适当的质量外，审查还可以用来检测在开发和维护阶段内发生的质量变化。

- 使用提高软件质量的工具和技术
  - 好的软件分析、设计技术
    - 结构化技术
    - 面向对象技术，UML
  - 好的文档工具
    - Rose、PowerDesigner
  - 先进的软件体系结构
    - C/S、B/S
    - 基于构件的结构、基于总线的结构等
  - 软件复用
    - 文档复用
    - 程序复用
- 选择可维护的程序设计语言
- 改进程序的文档

	检查重点	检查项目	检查方法和工具
需求分析	对程序可维护性的要求是什么？例如，对于可使用性：交互系统的响应时间。	软件需求说明书 限制与条件，优先顺序 进度计划 测试计划	可使用性检查表
设计	程序是否可理解？ 程序是否可修改？ 程序是否可测试？	设计方法 设计内容 进度 运行、维护支持计划	复杂性度量、标准 修改练习 耦合、内聚估算 可测试性检查表
编码及单元测试	程序是否可理解？ 程序是否可修改？ 程序是否可移植？ 程序是否效率高？	源程序清单 文档 程序复杂性 单元测试结果	复杂性度量、90-10测试、自动结构检查程序 可修改性检查表、修改练习 编译结果分析 效率检查表、编译对时间和空间的要求
组装与测试	程序是否可靠？ 程序是否效率高？ 程序是否可移植？ 程序是否可使用？	测试结果 用户文档 程序和数据文档 操作文档	测试、错误统计、可靠模型 效率检查表 比较在不同计算机上的运行结果 验收测试结果、可使用性检查表

维护过程概况



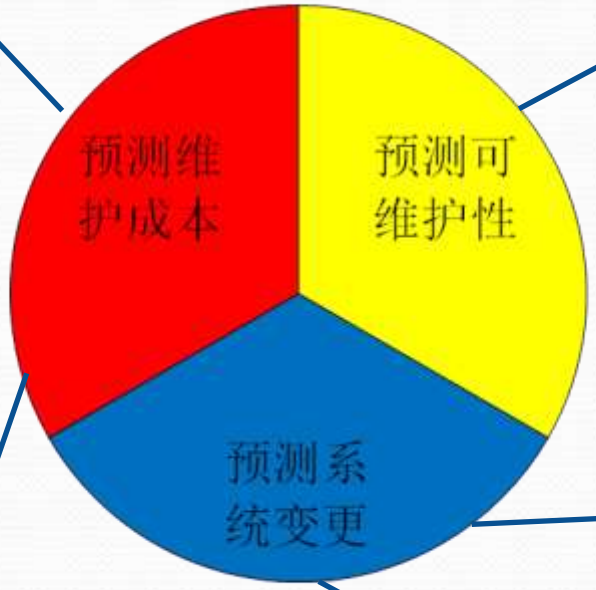
维护过程本质上是修改和压缩了的软件定义和开发过程，而且事实上远在提出一项维护要求之前，与软件维护有关的工作已经开始了。为了有效地进行软件维护，应事先就开始做组织工作。

① 首先建立维护的机构
② 申明提出维护申请报告的过程及评价的过程
③ 为每一个维护申请规定标准的处理步骤
④ 建立维护活动的记录保管，并规定复审的标准

维护预测

该系统整个生命周期的维护成本将是多少？

系统哪些部分的维护费用是最贵的？



该系统下一年度的维护费用将是多少？

能预测到的变更请求有多少？

系统哪些部分最容易受到变更请求的影响？

软件维护的副作用

- 所谓副作用是指因修改软件而造成的错误或其它不希望发生的情况。
- 软件维护的副作用主要有：
  - 编码副作用
  - 数据副作用
  - 文档副作用
- 预防的措施主要是程序模块化，信息隐蔽，高内聚、低耦合等。面向对象分析设计中使用封装、抽象、继承等办法。

软件进化

4 软件再工程

4.1 什么是再工程

- 软件再工程（software reengineering）关心的是通过对遗留系统改造使它们的可维护性得到提高。
- 一般软件功能不改变，系统的体系结构保持不变。
- 再工程包括：
  - 对系统重新建立文档
  - 组织并改造系统
  - 用一种更先进的程序语言转换系统
  - 修改或更新系统的体系结构
  - 修改或更新系统的数据

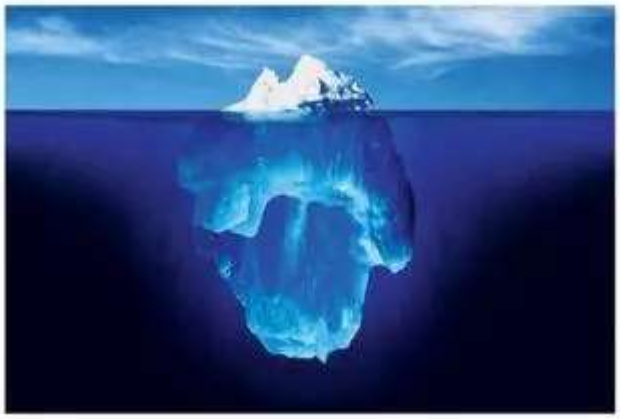


4.2 再工程的活动

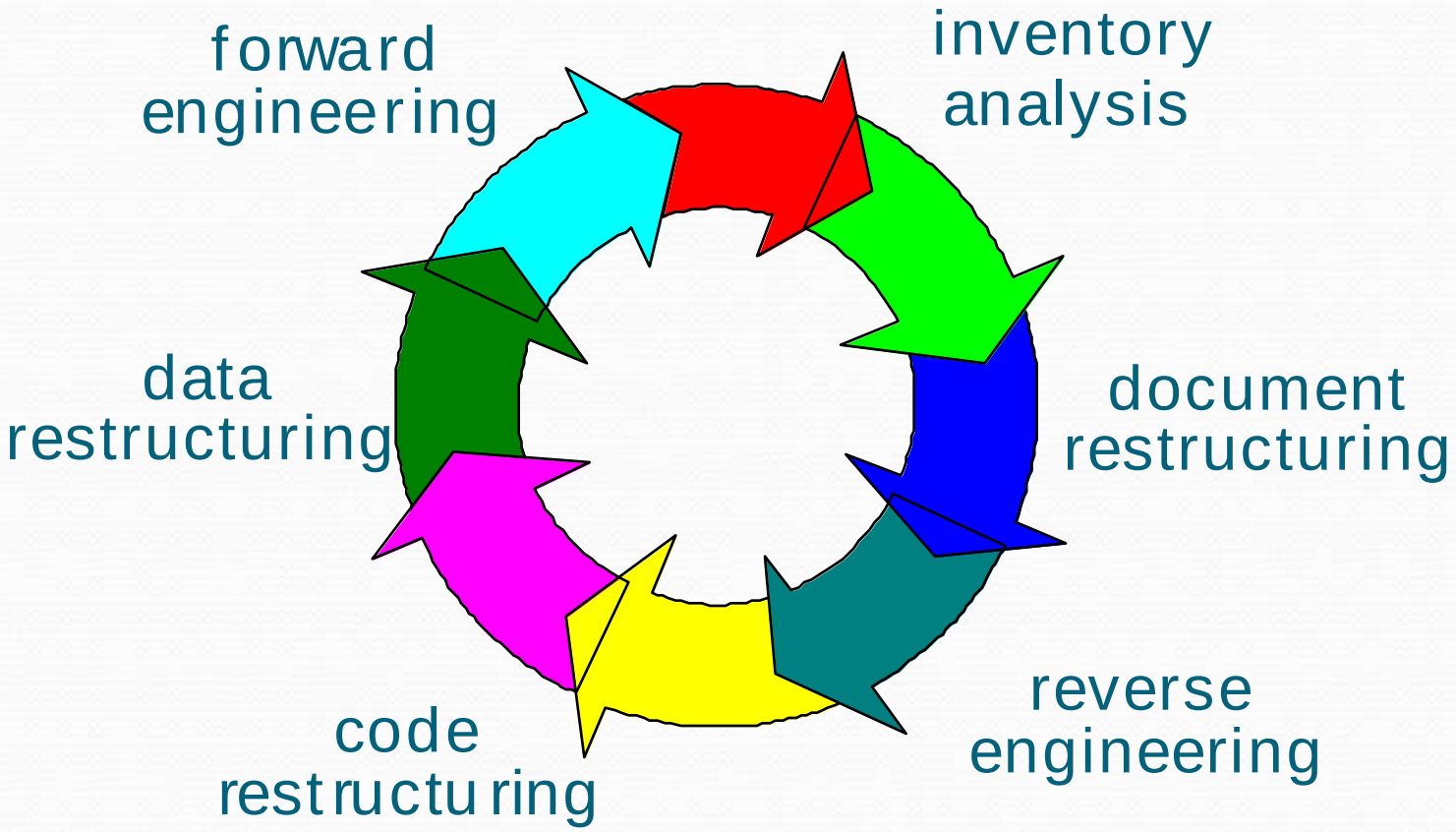
- 再工程与新软件开发的重要差别是在活动的起点上，再工程不是从写系统描述开始，而是将旧系统作为新系统的描述。
- Chikofsky和Cross（1990）称传统的开发过程为正向工程(forward engineering)以区别于软件再工程 (reengineering)。
- 对软件再工程的强调是源于多年来不断形成的软件维护“冰山(iceberg)”

- **再工程过程中的活动**：

①源代码转换(code conversion)
②逆向工程(reverse engineering)
③程序结构改善(architecture improvement)
④程序模块化(program modularization)
⑤数据再工程(data reengineering)



Software Reengineering



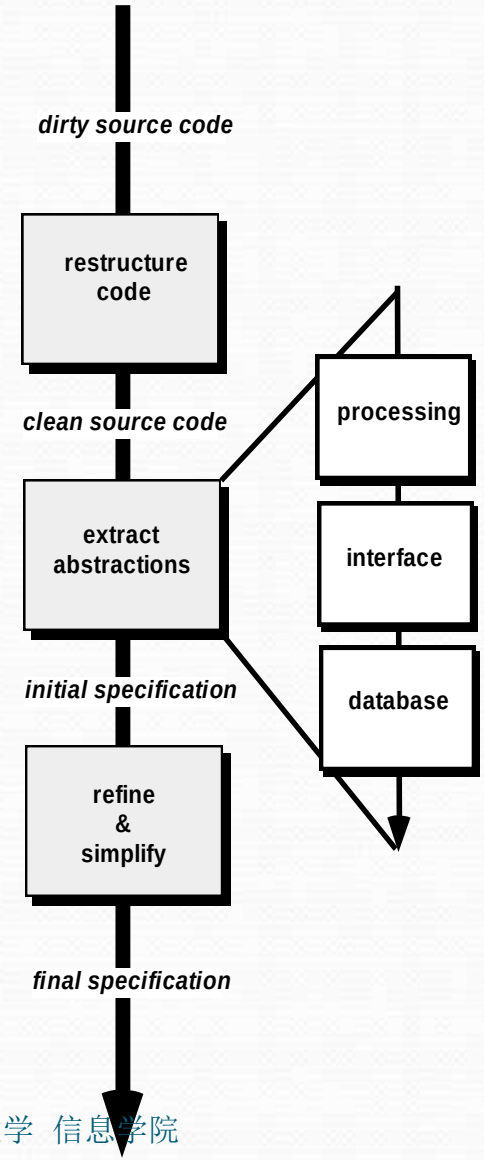
Inventory Analysis

- build a table that contains all applications
- establish a list of criteria, e.g.,
  - name of the application
  - year it was originally created
  - number of substantive changes made to it
  - total effort applied to make these changes
  - date of last substantive change
  - effort applied to make the last change
  - system(s) in which it resides
  - applications to which it interfaces, …
- analyze and prioritize to select candidates for reengineering

Restructuring

- Document Restructuring
  - options range from doing nothing to regeneration of all documentation for critical system
- Reverse Engineering
  - the intent here is to achieve design recovery
- Code Restructuring
  - rebuild spaghetti bowl code
- Data Restructuring
  - data architecture

Reverse Engineering



Forward Engineering

1. The cost to maintain one line of source code may be 20 to 40 times the cost of initial development of that line.

2. Redesign of the software architecture (program and/or data structure), using modern design concepts, can greatly facilitate future maintenance.

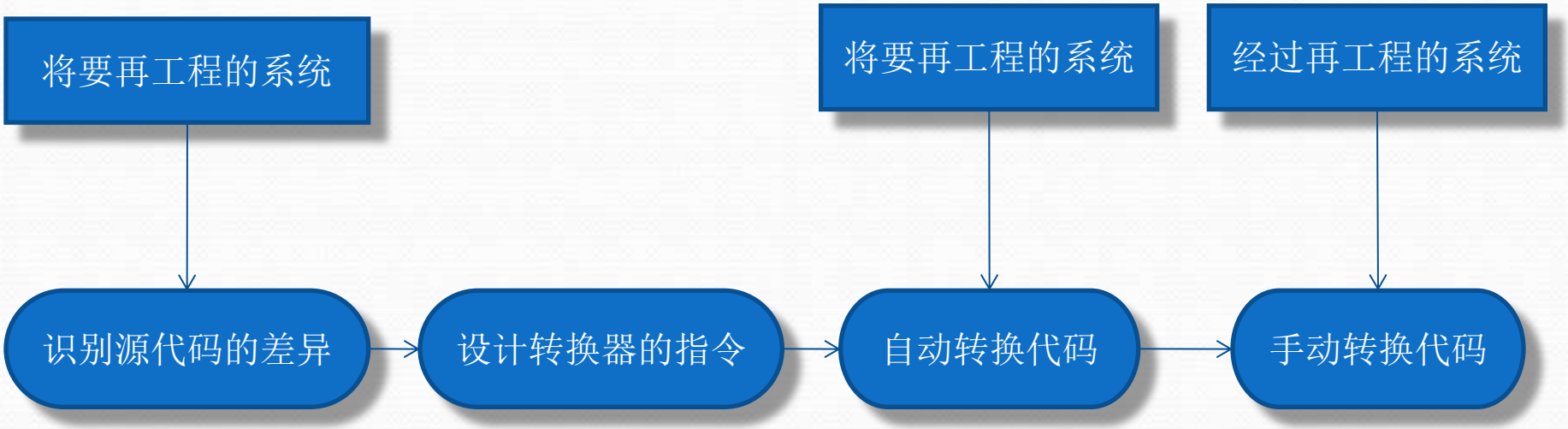
3. Because a prototype of the software already exists, development productivity should be much higher than average.

4. The user now has experience with the software. Therefore, new requirements and the direction of change can be ascertained with greater ease.

5. CASE tools for reengineering will automate some parts of the job.

6. A complete software configuration (documents, programs and data) will exist upon completion of preventive maintenance.

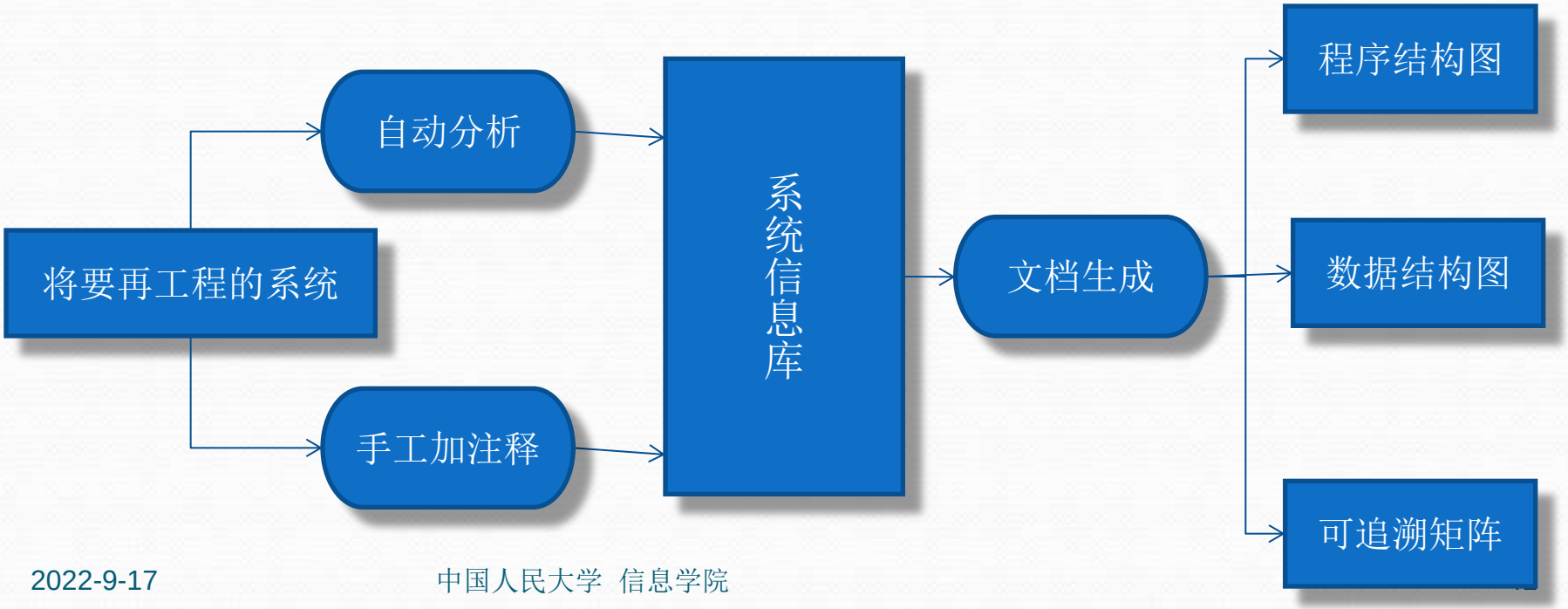
程序转换



- 软件再工程的一个最简单的形式是程序转换，将以一种语言写的源代码自动转换为另一种语言的源代码，程序本身的结构和组织没有发生变化。
- 目标语言可以是原先使用的语言的新版本，或者是一种全新的语言。
- 只有存在源代码自动转换器，能对大部源代码自动进行转换时源代码转换才是经济的。源语言中的结构可能在目标语言中没有直接的等价形式，就需要手工完成系统的代码转换。

4.3 逆向工程

- 逆向工程是复原软件的描述和设计目标的软件分析过程。
- 软件源代码可以作为逆向工程过程的输入，没有源代码时，只能用可执行代码开始。
- 软件再工程过程借助逆向工程来复原程序设计，以帮助工程师在重组系统的结构之前理解程序。



4.4 数据再工程

- 对数据结构的分析和重组过程，有时还包括对系统中的数据值的分析过程，被称为数据再工程。
- 原则上讲，如果系统的功能没有发生改变，那么数据再工程就没有必要。
- **数据再工程的原因**

①数据退化

- 随着时间的推移，数据的质量在逐步下降；
- 对数据的变更引入了错误，数据产生了的副本；
- 客户的环境变化，但变化没有正确地反映到数据中。

②程序内固有的限制

③体系结构进化

数据的不一致问题

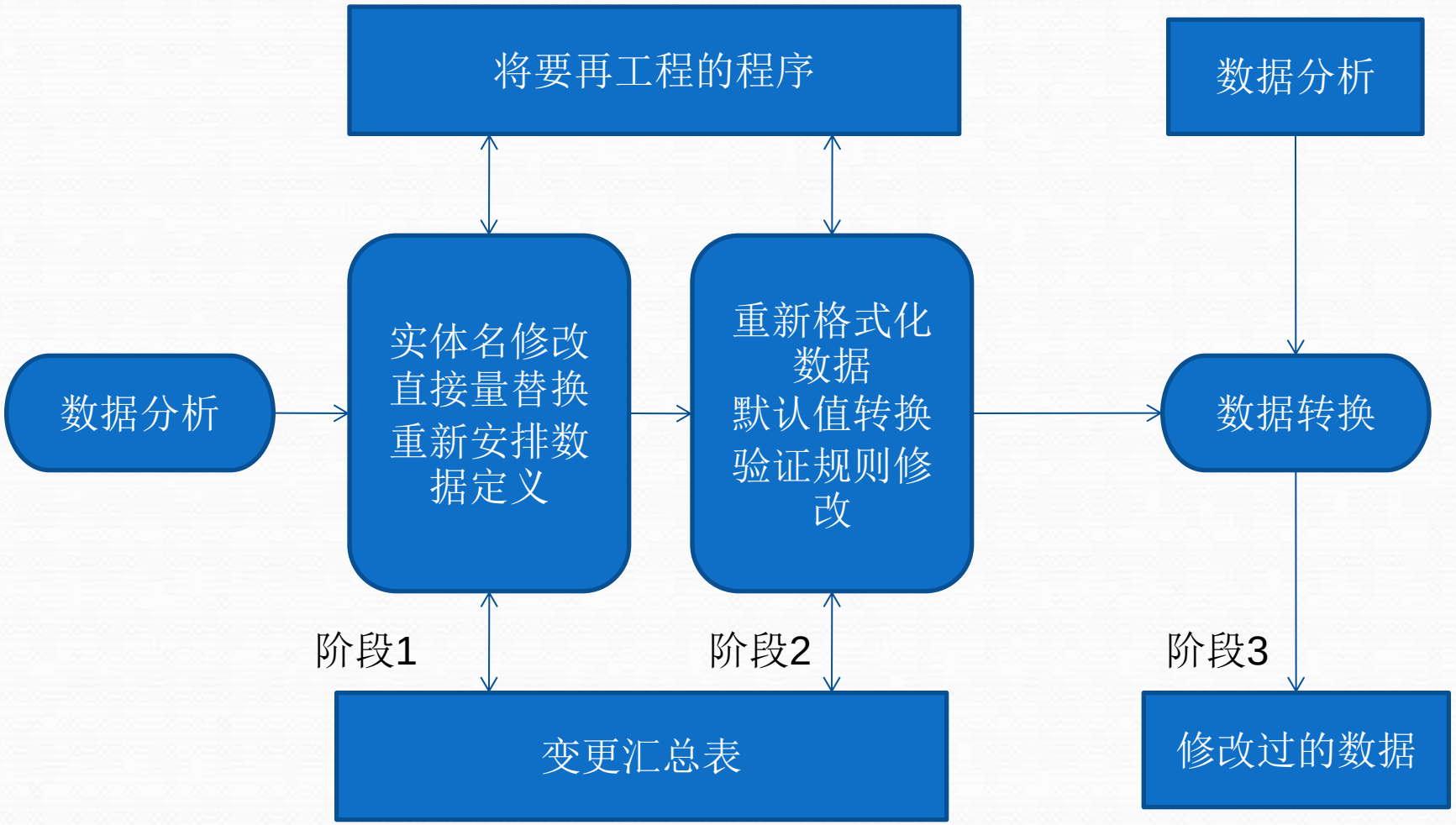
- 不一致的默认值
- 不一致的单位
- 不一致的验证规则
- 不一致的表示语义
- 不一致的负值处理
- 不一致标准
- 不一致的计算精度



数据再工程的方法

- 数据清理
  - 对数据记录和值进行分析以改善它们的质量。
  - 重复之处被删除，向所有记录提供一致的格式。
- 数据扩展
  - 通过程序和数据的再工程，删除数据处理上的限制。
  - 需要变更程序来增加域长度、修改表的上限等。
  - 数据本身需要重写和清理来反映程序的变化。
- 数据迁移
  - 这可能是在同一DBMS的不同版本之间，也可能是在不同DBMS之间进行。

数据再工程过程


将要再工程的程序
数据分析
数据分析
实体名修改
直接量替换
重新安排数据定义
重新格式化数据
默认值转换
验证规则修改
数据转换
阶段1
阶段2
阶段3
变更汇总表
修改过的数据

软件重构

- 软件重构（Software Refactoring）是指在不改变软件的功能和外部可见性的情况下，为了改善软件的结构，提高软件的可读性、可扩展性和可重用性而对其进行的改造。
- 简而言之，重构就是改进已经写好的软件。
- 重构可以是软件迭代开发的一部分， 在迭代周期中通过重构代码来改善代码的质量。
- 测试保证可以重构没有改变代码的外部行为。
- 重构作为软件维护技术，可以既不修正错误，又不增加新的功能性。只是用来提高代码的可读性或者改变代码的结构和设计，进而提高软件的可维护性。

软件进化

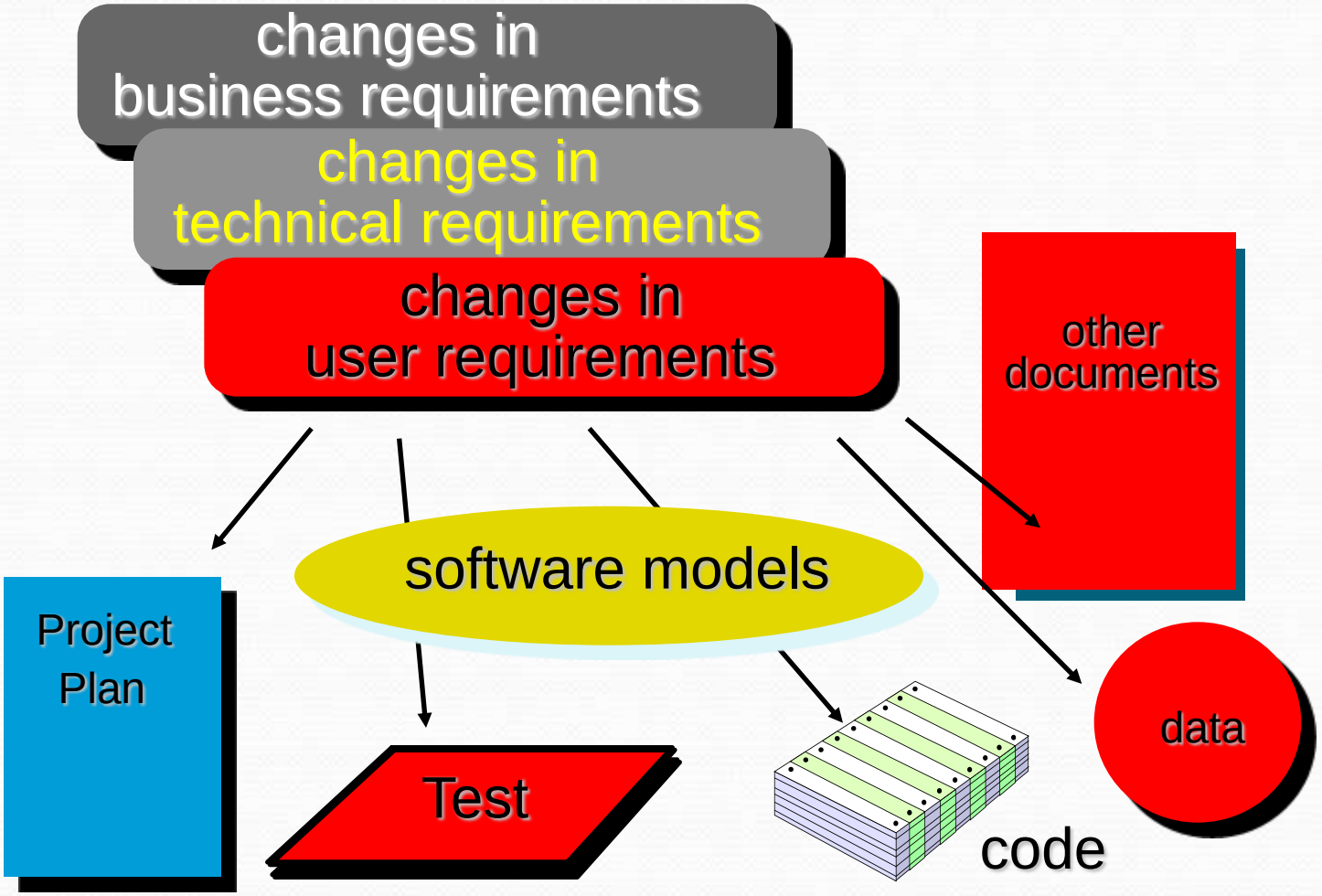
5 软件配置管理

The “First Law”

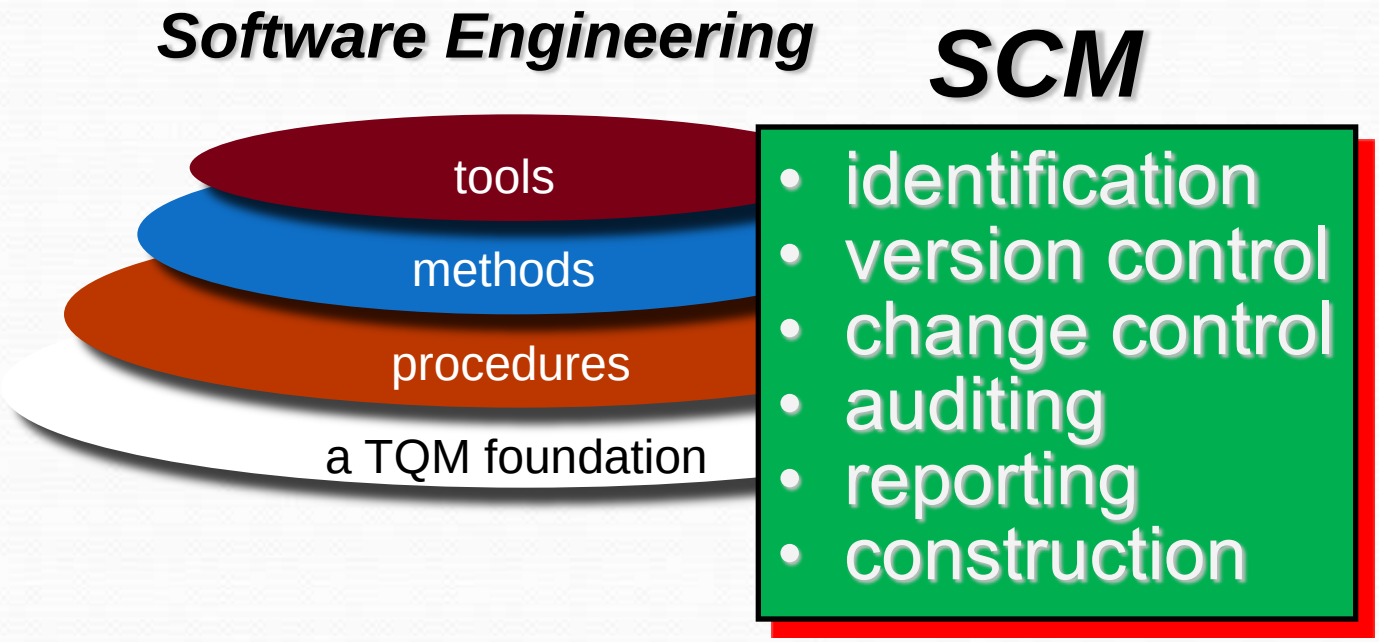
No matter where you are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle.

Bersoff, et al, 1980

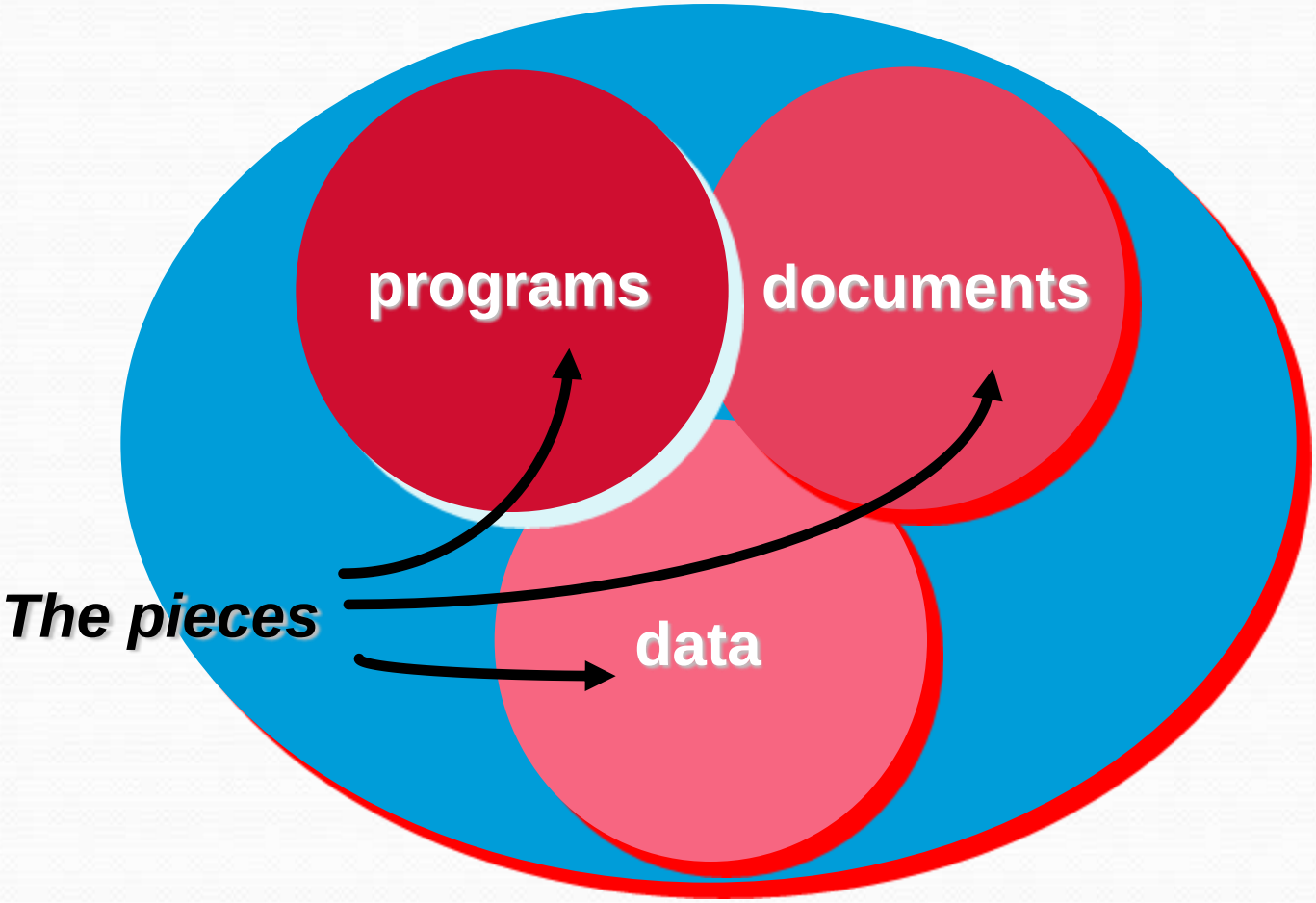
What Are These Changes?



Change & SCM


Software Engineering
tools
methods
procedures
a TQM foundation
SCM
• identification
• version control
• change control
• auditing
• reporting
• construction

The Software Configuration



5.1 什么是软件配置管理

- 软件配置管理（software configuration management）
- 配置管理在其它领域已有广泛的应用。

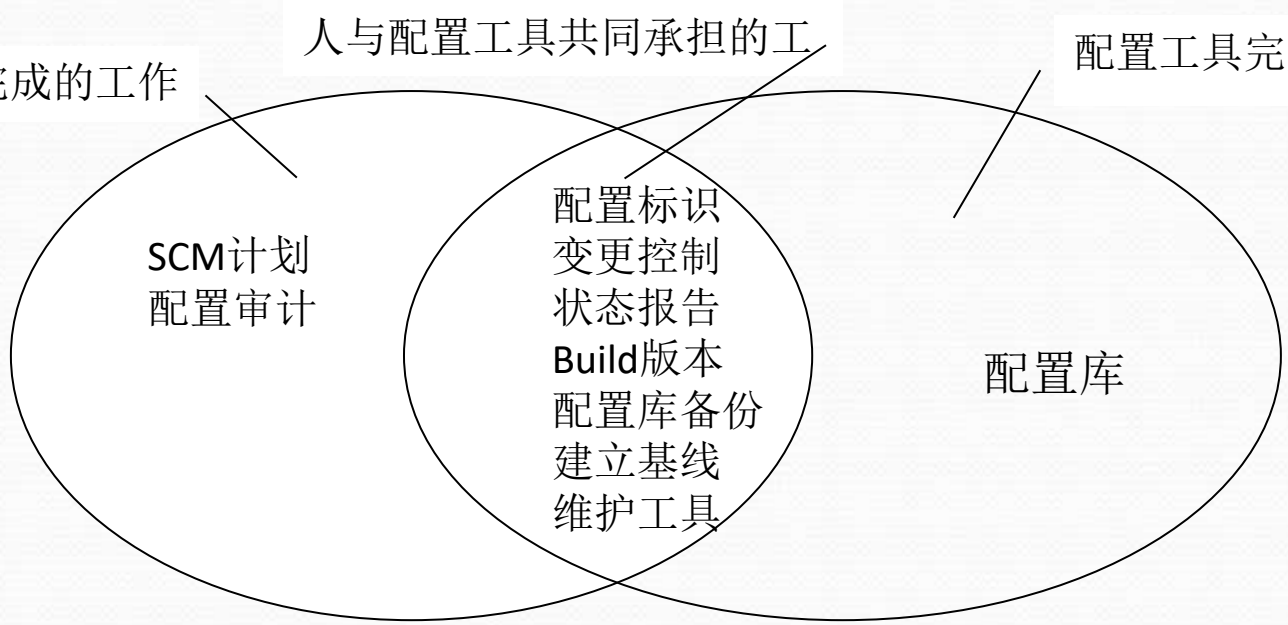


图3 人与配置工具的任务分配

软件配置管理定义

ISO 9000实施指南：

- 软件配置管理是一个管理学科，它对配置管理项的开发和支持生存周期给予技术上和管理上的指导。配置管理的应用取决于项目的规模、复杂程序和风险大小。

GB/T 11457: 1995软件工程术语解释：

- 标识和确定系统中配置项的过程，在系统整个生存周期内控制这些项的投放和更动，记录并报告配置的状态和更动要求。验证配置项的完整性和正确性。

Babich认为：

- 软件配置管理能协调软件开发，使得混乱漫漫减少到最小。软件配置管理是一种标识、组织的控制修改的技术，目的是最有效地提高生产率。

配置管理的作用：

- 软件配置管理负责软件的标识变更、控制变更和发布变更；
- 建立和维护软件产品在整个软件生存周期中的完整性、一致性和可追溯性；
- 使各有关人员所见所用的都是有效版本。

什么是软件配置项

（software configuration item）

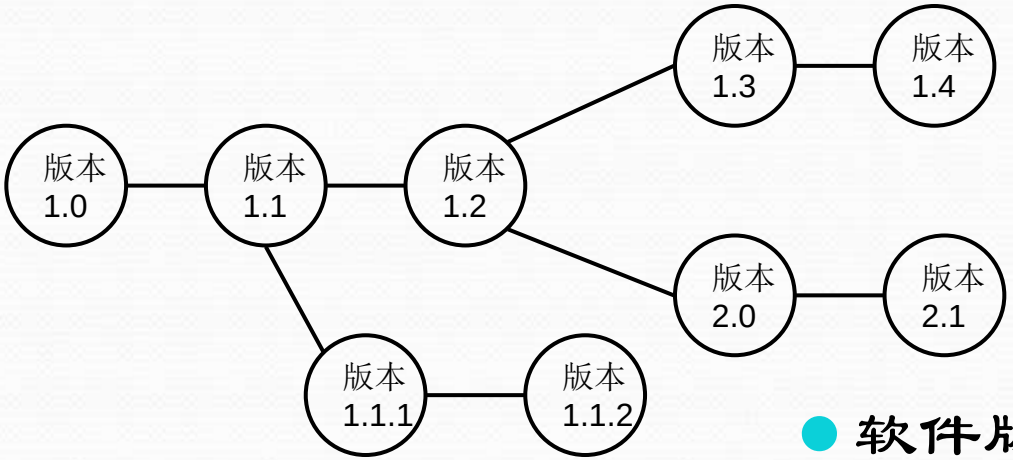
- 含义：
  - 为了配置管理的目的而作为一个单位来看待的硬件和（或）软件成分。
- 可包括：
  - 计算机可执行源代码、目标码、数据库、计算机不可执行的文档、源程序清单、测试用例等。

配置管理的产品

- 按ISO 9000实施指南规定：
  a. 与合同、源代码、过程、计划和产品有关的文档及数据；
  b. 目标代码和可执行代码；
  c. 相关产品，包括：软件工具、包括库在内的可复用软件，外购软件，顾客提供的软件。
- 软件配置：所有以上这些产品在不同时期，出于不同要求的组合，如针对不同硬件环境及软件环境的组合。

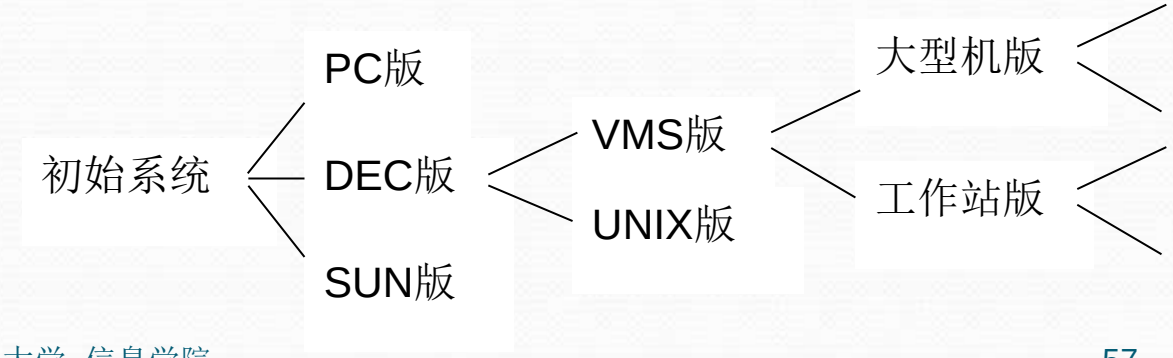
配置标识

- 用“版本”来定义配置项的演化阶段。



- **软件版本举例**

按软件运行的环境来标识版本。



5.2 软件配置管理任务

1) 制订配置管理计划
2) 确定配置标识
3) 进行配置控制，实施变更管理
4) 记录并报告配置状态
5) 进行配置评价
6) 实施发布和交付



(1)配置管理计划

- ISO 9000-3的配置计划内容：
  - 配置管理组织的职责
  - 配置管理活动
  - 使用的配置管理工具、技术和方法
  - 何时将配置项置于配置管理之下

(2)配置标识的任务

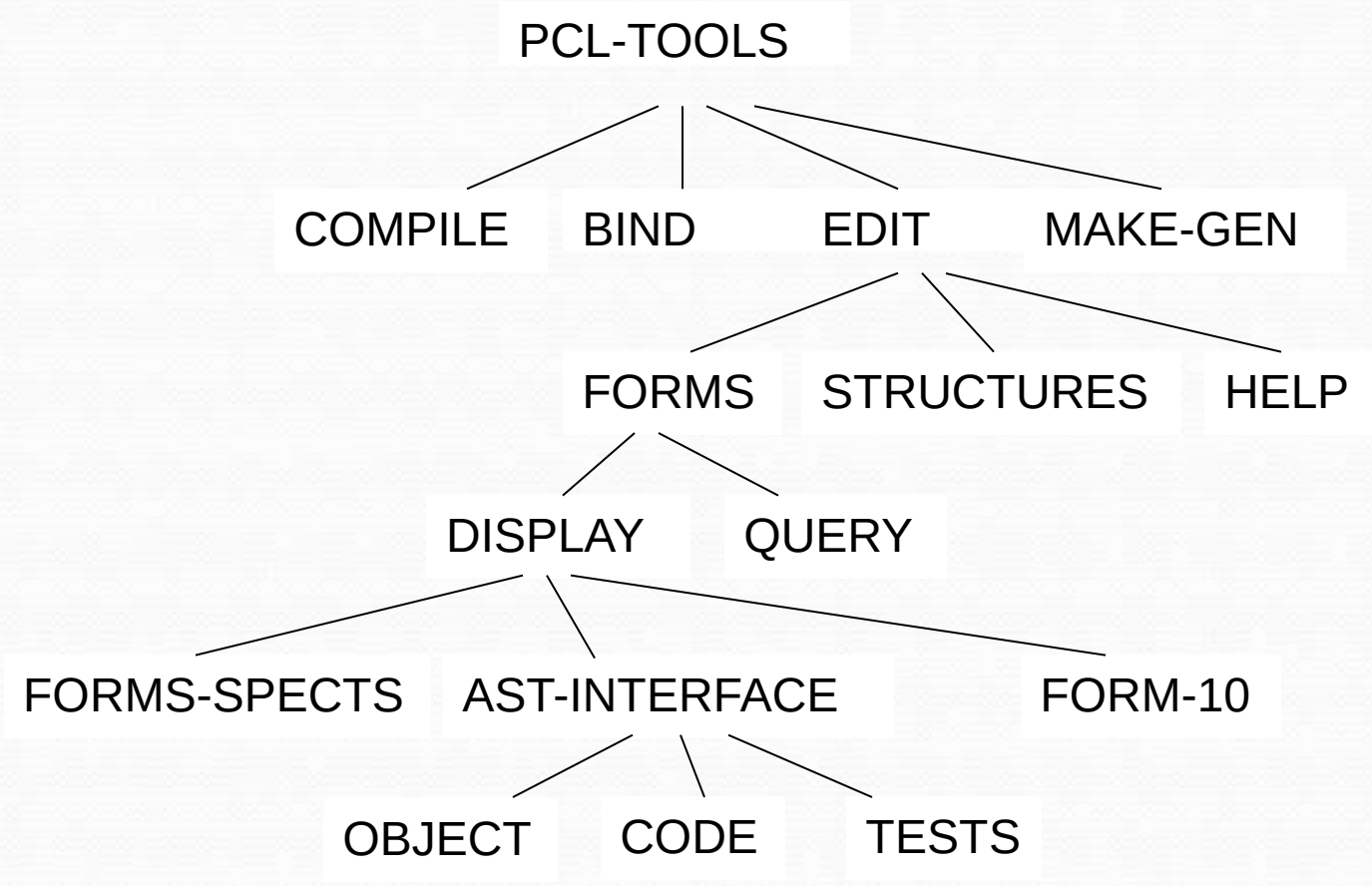
①确定配置项

- 大型软件开发过程中会产生数以千计的文档，这此文档经常变更，但它们又是下一阶段工作的依据。同时会出现许多计划书、建议书、会议记录、备忘录。配置管理的一项重要任务是做出选择，决定哪些受控，称之为**配置项**。

②制订命名规则

- 要求：唯一性、反映文档间的相互关系（可追溯）。通常采用树状（层次式）命名规则。

例：树状命名


PCL-TOOLS
COMPILE
BIND
EDIT
MAKE-GEN
FORMS
STRUCTURES
HELP
DISPLAY
QUERY
FORMS-SPECTS
AST-INTERFACE
FORM-10
OBJECT
CODE
TESTS

(3)变更管理

①软件系统变更不可避免，变更管理任务

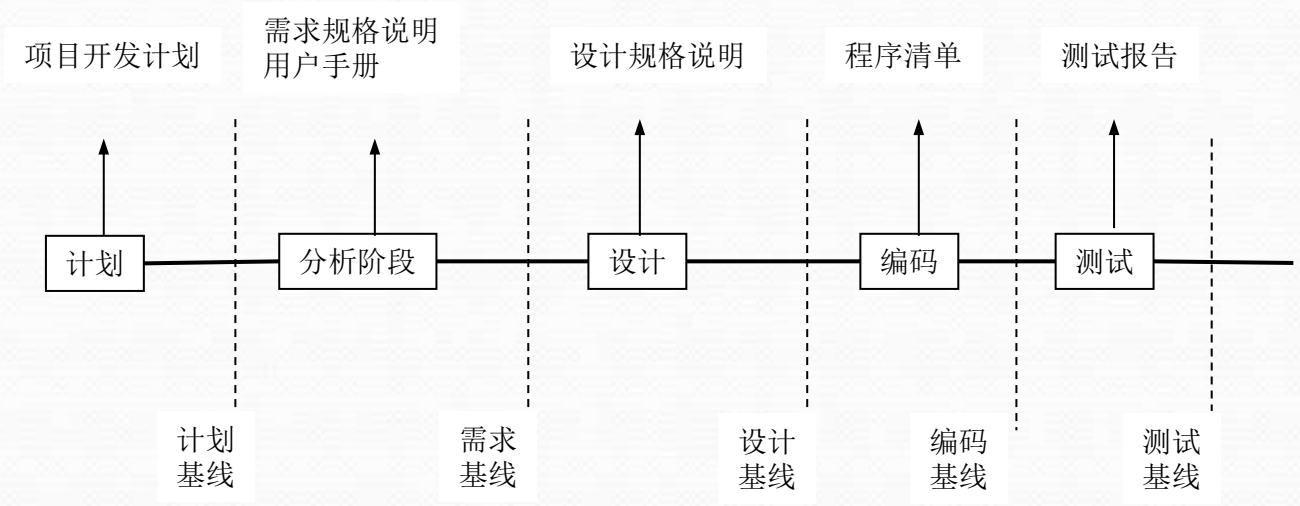
- 分析变更：成本——效益、技术；
- 记录变更及追踪变更；
- 保证变更在受控下进行。

②基线和变更控制

- 基线（base line）是一个配置项在生存周期内某一特定时间正式标明、固定并经正式批准的一个版本。
- 基线中的特定时间一般是软件生存周期各阶段末尾的特定点，也称为里程碑（milestone）。
- 基线使软件开发工作在这些点上断开，便于检验和肯定阶段成果。
- 基线不可以随意修改，修改必须按规定的程序进行。

软件配置基线

- 基线(baseline)标志一个开发阶段的结束，形成相对稳定的交付物或相关的SCI，同时作为下开发阶段的基础，或输入项。
- 对于交付物，基线是版本控制的基础，可回滚，也是变更控制的基础。



举例：选用瀑布开发模型的基线设置方案

③变更管理过程

1) 提交变更请求表CRF
2) 分析变更请求
3) IF 变更是否成立 THEN
4) 估计变更如何实现
5) 估算变更成本
6) 将CRF送交CCB
7) IF 变更获准 THEN
8) REPEAT
9) 实施变更
10) 记录变更
11) 将变更的软件提交QA审查
12) UNTIL 软件质量达到要求
13) 由CM人员生成系统的新版本
14) ELSE
15) 拒绝变更请求
16) ELSE
17) 拒绝变更请求

变更请求表（Change Request Form，CRF）

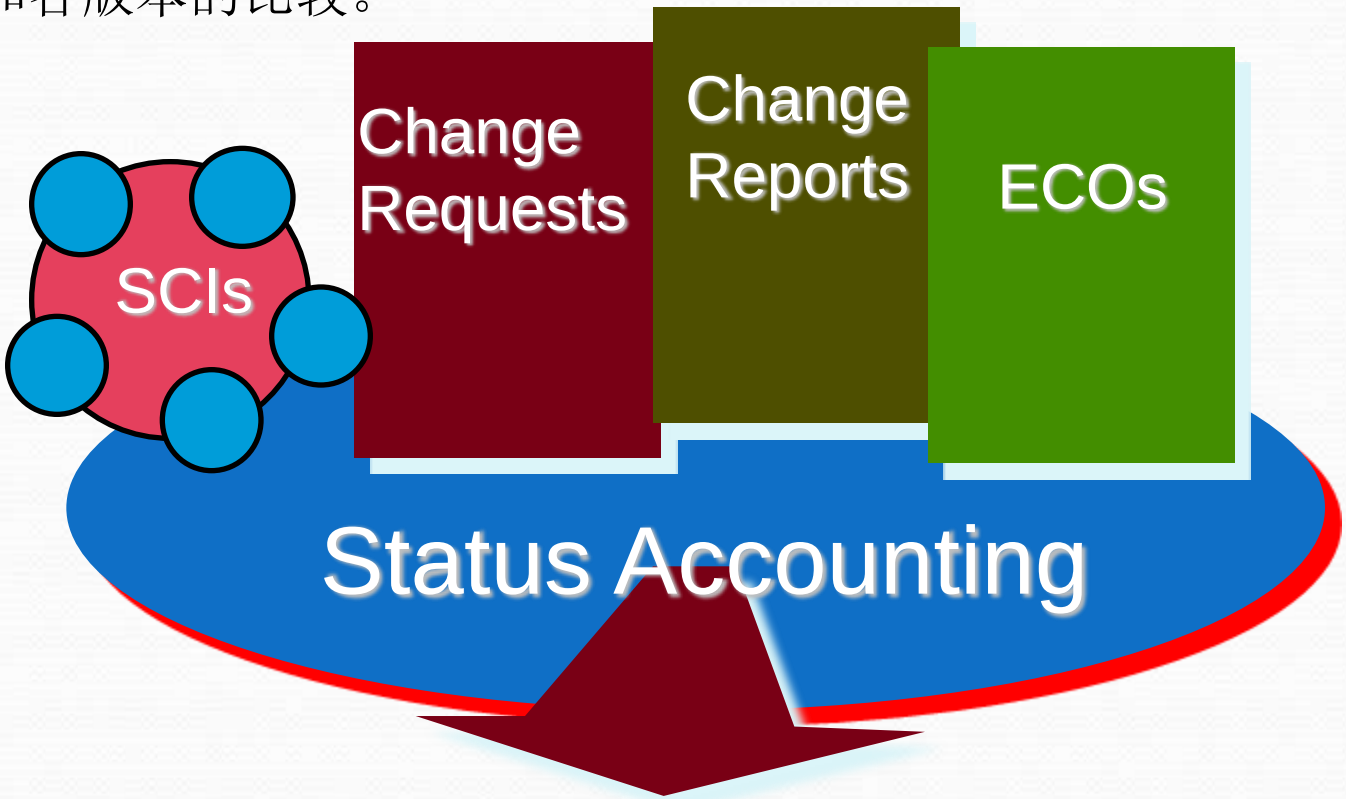
变更控制委员会（Change Control Board，CCB）

质量保证（Quality Assurance，QA）

配置管理（Configuration Management，CM）

(4)配置状态记录和报告

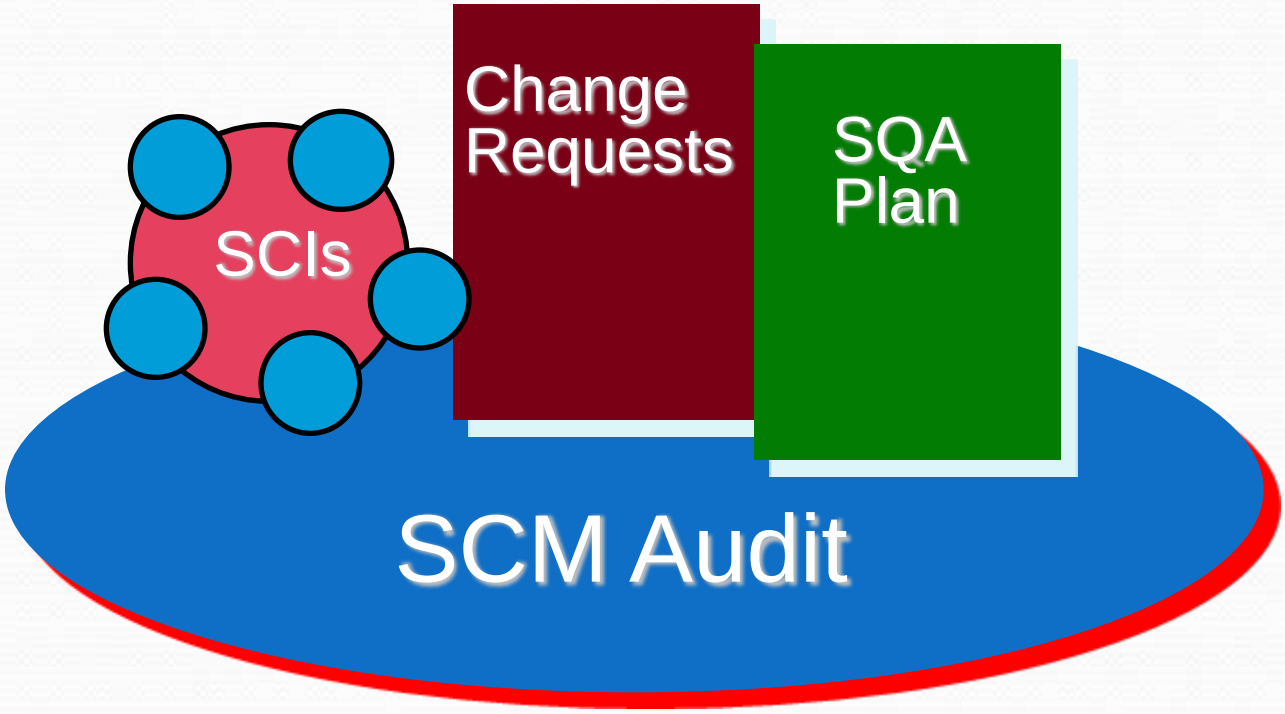
- 记录和报告所有软件项的状态和历史。
- 状态报告应包含有关项的更改数、最新的软件项版本、发行标识、版本号和各版本的比较。



Status Accounting

(5)配置评价

- 进行功能配置审计，确保软件项的功能完整性；
- 进行物理配置审计，确保软件项的物理完整性。



Auditing

例：配置审计检查列表

- 发布审核
  1、发行文档是否已经明确规定了发行范围
  2、所有已经发现的缺陷或隐含差错是否均作记录
  3、是否有文档说明了再次发行的环境
  4、是否有文档说明了部件极其发行部件的版本情况
  5、是否所有发行的配置项相互之间协调一致
  6、发行的产品是否从配置仓库取出的合适版本
- 基线库和配置项审核
  1、基线库是针对每一个软件配置管理计划制订的吗？
  2、配置项均已放入适当的极限库吗？
  3、需要的配置项能否在基线库中找到吗？
  4、配置项的命名是根据软件配置管理计划规定的命名方法吗？
  5、配置项的版本号是根据软件配置管理计划规定的吗？
  6、所有的配置管理计划是根据配置管理计划规定的条件纳入基线的吗？
  7、配置项是否正确标识、确定版本并记载变更历史吗？

例：配置审计检查列表（续）

- 实施变更的审核
  1、所有的变更申请均已经处理了吗？
  2、变更请求是否列入所有要做变更的配置项
  3、在变更请求中所有被标识要变更的配置项都已经作了变更？都作了质量控制？
  4、任何配置项的两个版本相互交换能查出来吗？
  5、对配置项进行变更之前已经将变更请求文档化？
  6、对配置项进行变更之前，变更申请是否已经分析、评价和批准？
- 其他方面审核
  1、配置仓库是否作了备份？
  2、是否有适当的保密或授权制度来保证只有经过授权的团队人员才能做检入和检出操作？

(6)发布和交付

① 版本管理是对系统不同版本进行标识和跟踪的过程。

- 版本标识的目的是便于对版本进行检索和跟踪，显示各版本的关系；
- 一个版本是软件系统的一个实例，在功能和性能上与其它版本有所差别，或修正了前一版本的不足；
- 有些版本功能上等价，但分别适用于不同的硬件装置；
- 如果两处版本只有少量差异，则互相称为变体（Variant）。

② 系统发布是分配给客户一个版本，每次系统发布都应有新的功能或是针对不同的硬件平台。

- 通常软件系统的版本数要比发布的多，因为有的版本并未发行，例如，有的版本仅供公司内部使用或专供测试等。
- 一次发布不仅是一个可执行程序，或一套程序。

以SCM为中心的项目组织结构

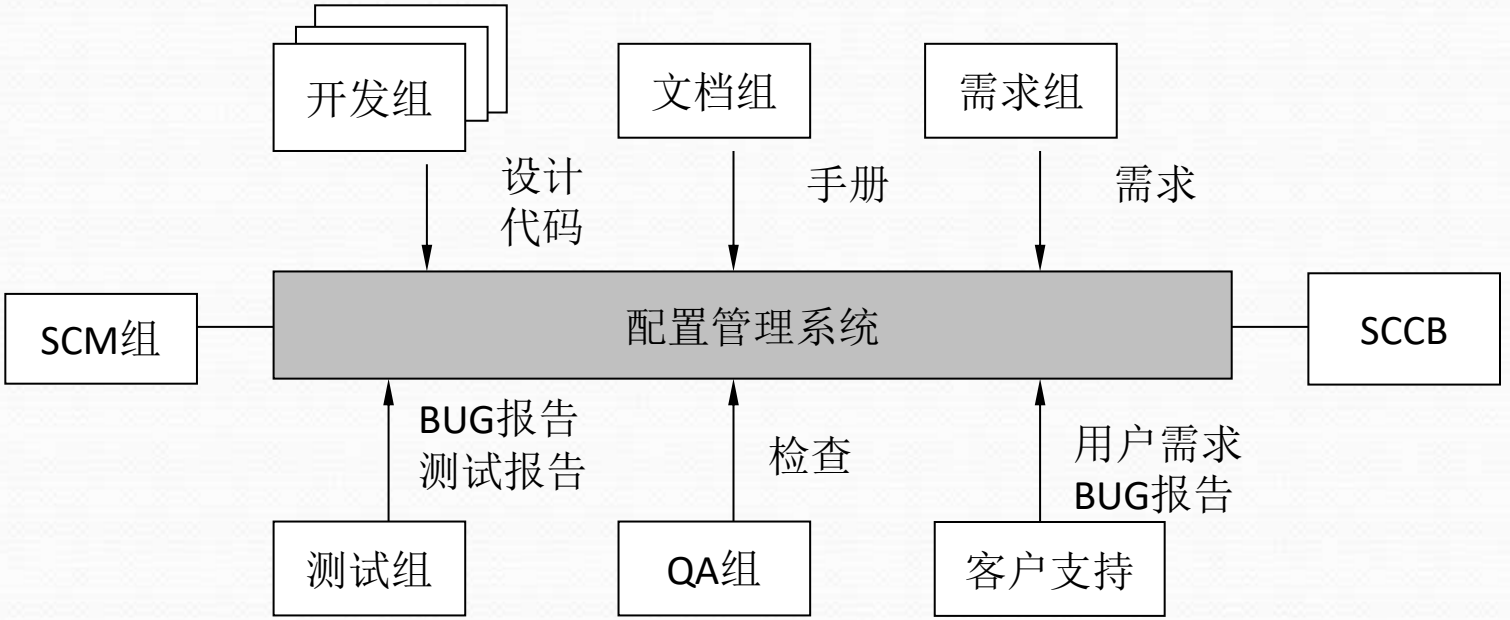


图1　以SCM为中心的项目组织结构

5.3 配置数据库

- 作用
  - 用于记录与配置相关的所有信息；
  - 评价系统变更的后果；
  - 提供配置管理过程的管理信息。
- 三类库
  - 开发库（Development Library）
  - 受控库（Controlled Library）
  - 产品库（Product Library）



配置与开发过程的关系

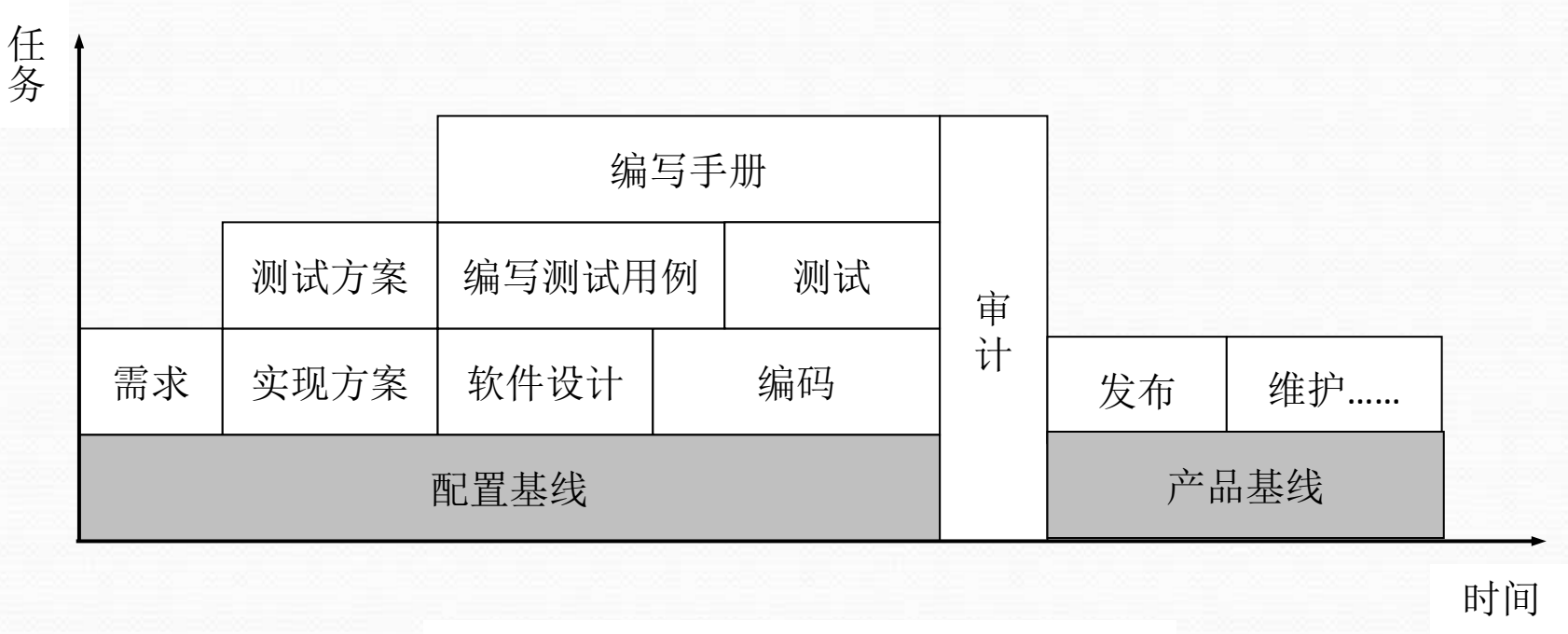


图2 配置与开发过程的关系

10种常见SCM工具

① VSS-Visual Source Safe,Microsoft开发
② CVS-Concurrent Versions System，开源工具
③ SVN-CollabNet Subversion，在CVS基础上，由CollabNet开发，开源工具
④ ClearCase，由IBM Rational Software开发
⑤ PVCS，Serena公司开发
⑥ MK Source Integrity，由MKS公司开发
⑦ BitKeeper，由BitMover公司开发
⑧ AccuRev，由AccURev公司开发
⑨ Perforce，由Perforces Software公司开发
⑩ CM Synergy，原名Continuus，由Telelogic开发

软件进化

6 DevOps

全球软件产业正在向服务化转型

软件服务化需要软件企业关注长尾效应、专注业务升级、降低开发与维护成本，要求面向服务的方式构建软件，“多、快、好、省”地持续推出新服务或者升级现有服务。

微软 Office：

- Office 2019 作为一个一次性的购买出售；
- Office 365 是一种订阅服务，确保用户始终拥有最新的 Microsoft 生产力工具。

据艾瑞咨询预测：

- 中国SaaS服务市场规模2021年将增长至654.2亿元；
- 2018 年中国SaaS企业软件市场规模为243.5亿元，年复合增长40%多。

云成为软件的普遍承载方式

- 来自Gartner的相关观点：
  - 到2020年，云计算将成为最主流的IT形态；
  - 云计算已经成为增长最快的科技领域，整体增长速度为25%，而整体IT市场的增长率仅为1.1%；
  - 新一代的硅谷明星企业，比如Airbnb、Pinterest、Lfyt等都生长在云上。

新形势下企业面对多重挑战

交付频率高，研发周期短	跨地域协作多，部署发布复杂	可靠性与安全要求高
		
• 小特性1天交付一次，版本2周交付一次； • 快速交付、快速反馈； • 精益创业模式（MVP）。	• 跨地域沟通协作多、效率低； • 研发环境、类生产环境、生产环境不一致； • 多Region并行部署上线复杂； • 急需一站式开发、测试、部署、运维平台。	• 7X24小时运行，可靠性要求高； • 公有云服务，安全要求高； • 核心研发数据在传输与存储上存在风险；

DevOps是什么

DevOps的概念

维基百科， http://zh.wikipedia.org/wiki/DevOps
DevOps（Development和Operations的组合词）是一种重视“软件开发人员（Dev）”和“IT运维技术人员（Ops）”之间沟通合作的文化、运动或惯例。透过自动化“软件交付”和“架构变更”的流程，来使得构建、测试、发布软件能够更加地快捷、频繁和可靠。

DevOps专家的观点

- **DevOps打破开发与运维之间的孤岛**，积极协作。广义上指整个组织级的协作优化，包括IT、HR、财务、和公司供应商。约束理论告诉我们，需要优化整体而非单个“孤岛”。整体是与客户问题相关的业务，从精益角度来说，是整个价值链。

Patrick Debois，DevOps名词的发明和运动的推动者，DevOps Days创办人之一。

- **DevOps通常指的是新兴的专业化运动**，提倡开发和IT运维之间的高度协同，从而在完成高频率部署的同时，提高生产环境的可靠性、稳定性、弹性和安全性”。DevOps聚焦实现更快的上市时间，减少IT浪费，提高组织效率。

Gene Kim，DevOps领导者，《The Phoenix Project: A Novel About IT, DevOps》、《The Visible Ops Handbook》作者。

DevOps领先企业的观点

亚马逊

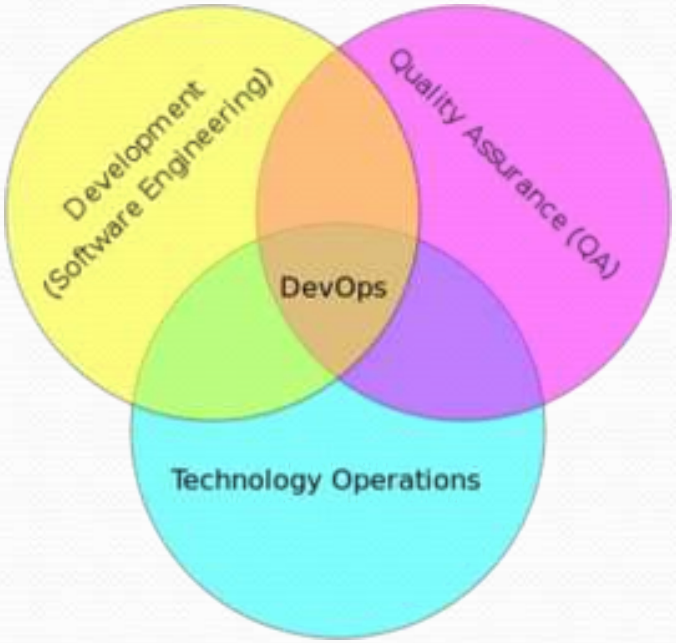
- DevOps侧重于改善协作，沟通，整合软件开发与IT运维。是一个总称，用来描述一种**理念、文化变革和模式转变**。
- DevOps帮助亚马逊**从软件开发敏捷走向业务和运维的敏捷**。
- 开发阶段，DevOps重点放在代码构建，代码覆盖，单元测试，打包和部署；运维阶段，在基础设施上聚焦环境分配，配置，编排和部署。

微软

- DevOps不仅是一项技术或工具集，也**是观念和文化的转变**。它结合人、流程、适当的工具，使应用程序生命周期更快，更可预测。
- 我们**应将DevOps视为过程而非目的**，应该选定适合范围的项目，增量式实施，通过这些项目展示成功，学习，并演进。

DevOps: Development和Operations的组合

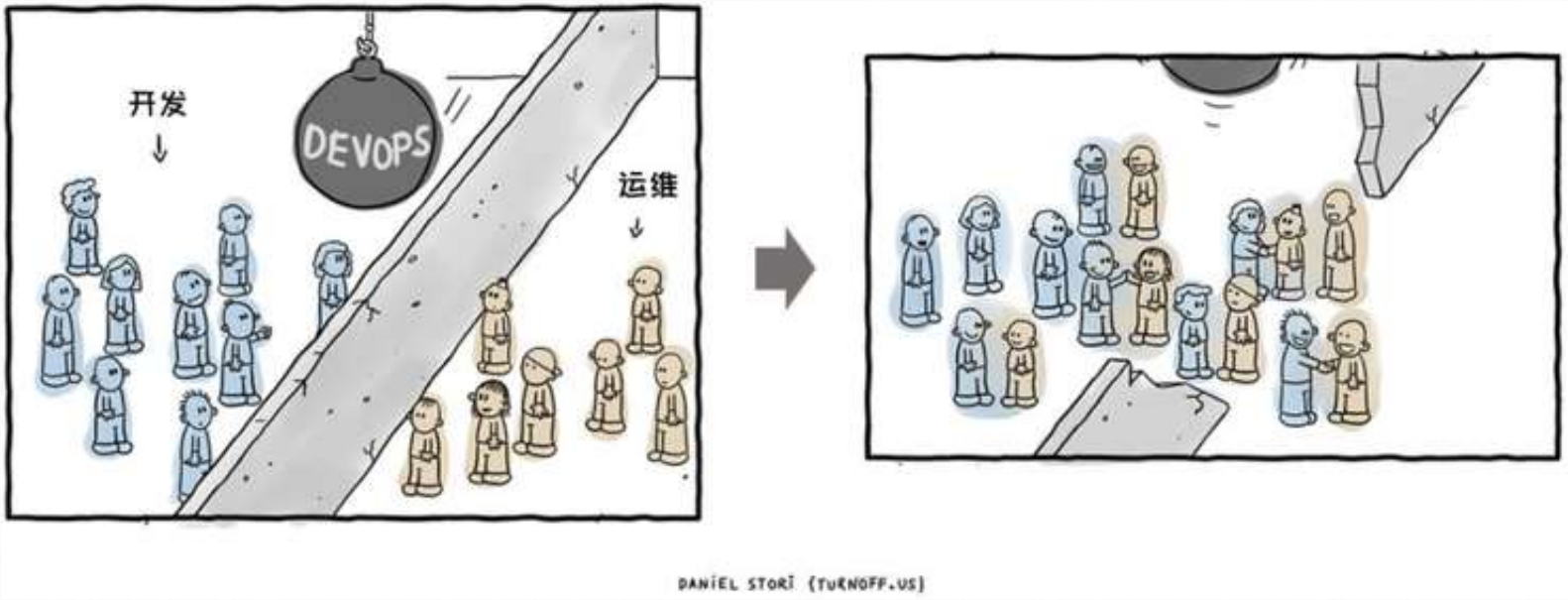
- DevOps是一种方法论，其中包含一系列基本原则和实践。
- DevOps是一组过程、方法与系统的统称，用于促进开发（应用程序/软件工程）、技术运营和质量保障（QA）部门之间的沟通、协作与整合。它的出现是由于软件行业日益清晰地认识到：**为了按时交付软件产品和服务，开发和运营工作必须紧密合作。**



DevOps的引入

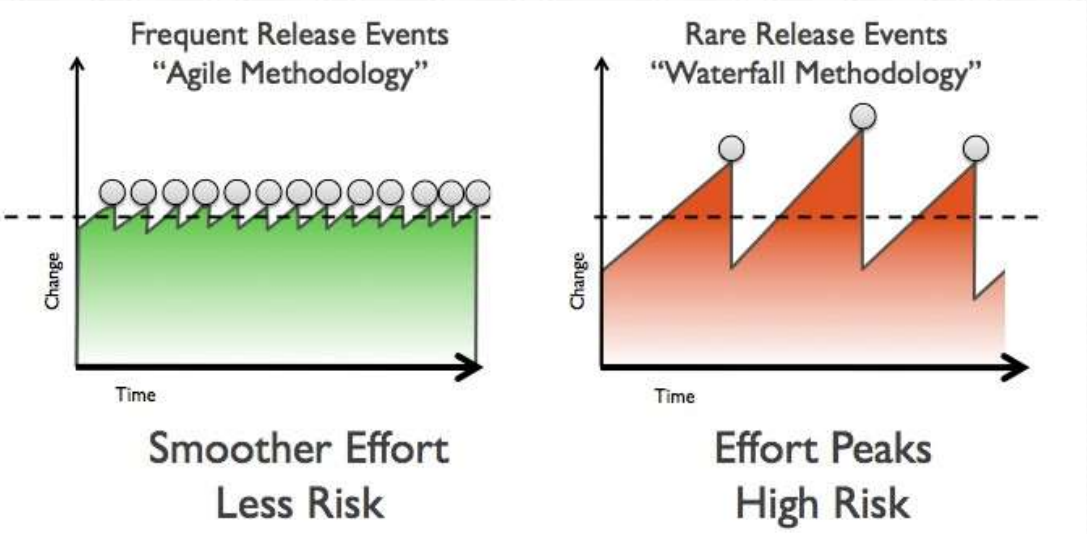
- 有一种观点认为，占主导地位的“传统”美国式管理风格（“斯隆模型 vs 丰田模型”）会导致“烟囱式自动化”，从而造成开发与运营之间的鸿沟。
- DevOps经常被描述为“开发团队与运营团队之间更具协作性、更高效的关系”。DevOps的引入能对产品交付、测试、功能开发和维护起到意义深远的影响。
- 以下几方面因素可能促使一个组织引入DevOps：
  - 使用敏捷或其他软件开发过程与方法
  - 业务负责人要求加快产品交付的速率
  - 虚拟化和云计算基础设施（可能来自内部或外部供应商）日益普遍
  - 数据中心自动化技术和配置管理工具的普及

希望解决的问题



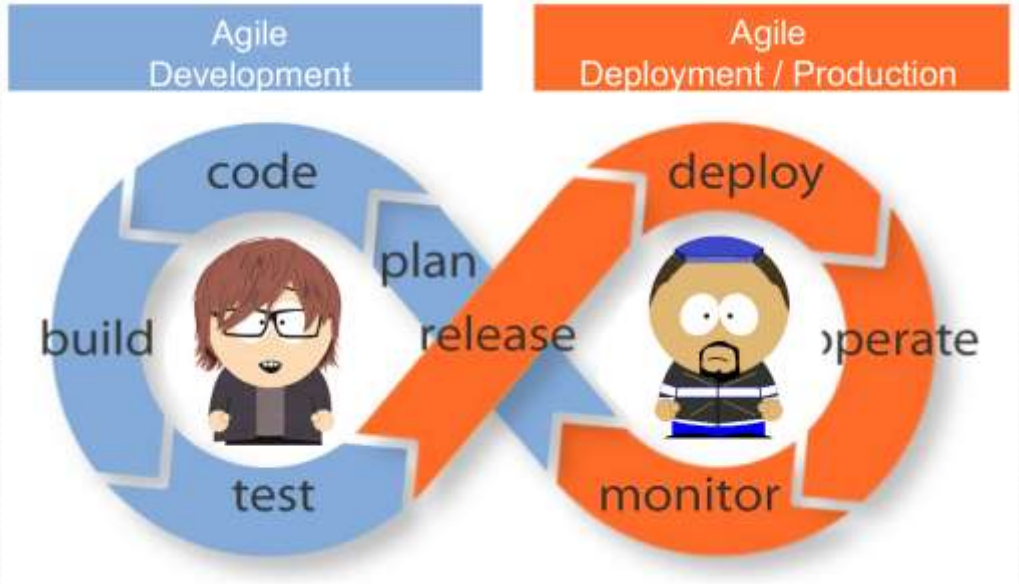
- 开发部门由功能性需求（通常与业务需求直接相关）驱动的，通常关注“频繁交付新特性”；而运营部门由非功能性需求（例如可获得性、可靠性、性能等）驱动的，则更关注IT服务的可靠性和IT成本投入的效率。
- 开发与运营两者目标不匹配，就在开发与运营部门之间造成了鸿沟，从而减慢了IT交付业务价值的速度。

快速持续交付



- 快速的部署其实可以帮助更快地发现问题，产品被更快地交付到用户手中，团队可以更快地得到用户的反馈，从而进行更快地响应。而且，DevOps小步快跑的形式带来的变化是比较小的，出现问题的偏差每次都不会太大，修复起来也会相对容易一些。

将敏捷扩展至生产端



- DevOps的共存主要是为了扩展敏捷开发实践，进一步完善软件变更在构建、验证、部署、交付等阶段中的流动，同时通过软件应用程序的全面所有权予力跨职能团队完成从设计到生产支持等各环节的工作。
- DevOps鼓励软件开发者和IT运维人员之间所进行的沟通、协作、集成和自动化，借此有助于改善双方在交付软件过程中的速度和质量。

持续交付的前提需求

- 对软件组件的开发和平台的供应和设置进行持续集成。
- TDD - 测试驱动的开发。这一点还有待商榷.....但始终还是需要面对：TDD是目前唯一能通过单元测试对代码和分支进行可接受程度覆盖的方法（单元测试使得问题的修复过程比集成测试或功能测试容易很多）。
- 代码审阅！ 至少要进行代码审阅.....如果能进行结对编程（Pair programming）当然就更好了。
- 软件的持续审计 - 例如使用Sonar。
- 在生产级环境实现功能测试的自动化。
- 更强大的非功能测试自动化（性能、可用性等）。
- 独立于目标环境的自动化打包和部署。

零停机部署

- 零停机部署（ZDD，Zero Downtime Deployment）可在不中断现有服务的情况下部署新版系统。
- ZDD的4种技术：
  - 功能开关（Feature Flipping）
  - 摸黑启动（Dark launch）
  - 蓝/绿部署（Blue/Green Deployment）
  - 金丝雀发布（Canary release）



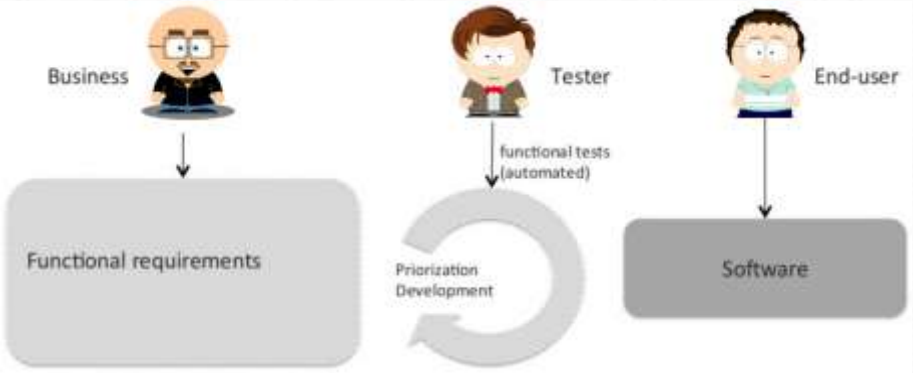
（续）

- 功能开关
  - 功能开关可供在软件运行过程中启用 / 禁用相应的功能。
  - 为生产版本提供一个能彻底禁用某项功能的配置，并只在对应功能彻底完工可以正常工作后才将该属性激活。
- 摸黑启动
  - 摸黑启动的目的在于通过生产环境进行负载模拟。
  - 在测试环境中，很难模拟出成百上千万用户规模的负载。如果不进行负载测试，就无法知道基础架构能否承受住最终面临的压力。
  - 此时并不需要模拟负载，而是可以实际部署这样的功能，然后看看在不影响可用性的前提下到底会发生什么。

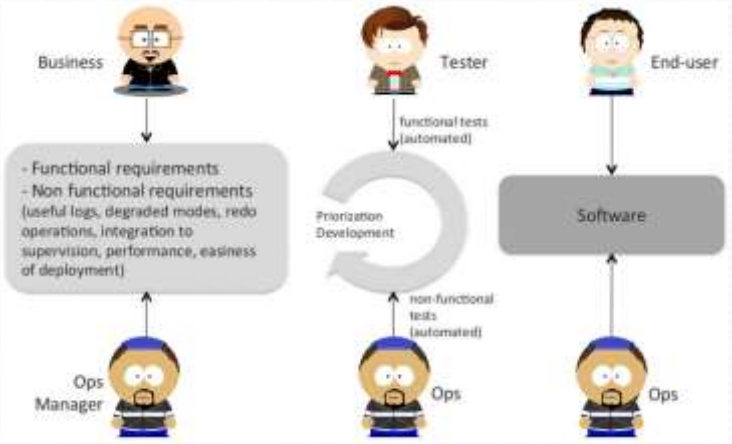
（续）

- 金丝雀部署
  - 一种增量发布方式。它的执行方式是在原有软件生产版本可用的情况下，同时部署一个新的版本。
  - 同时运行同一个软件产品的多个版本，用来测试生产中的软件，并将这一方式作为其可持续交付的一部分。
  - 需要软件针对配置和自动化部署进行特别设计。
- 蓝绿部署
  - 一种以可预测的方式发布应用的技术，目的是减少发布过程中服务停止的时间。
  - 需要准备两个相同的环境（基础架构），在蓝色环境运行当前生产环境中的应用，也就是旧版本应用。当你想升级时，在绿色环境中进行操作，即部署新版本应用，并进行测试。
  - 如果测试没问题，就可以把负载均衡器 / 反向代理 / 路由指向绿色环境了。随后你需要监测新版本应用，是否有故障和异常。如果运行良好，就可以删除蓝色版本使用的资源。如果运行出现了问题，你可以通过负载均衡器指向快速回滚到蓝色环境。

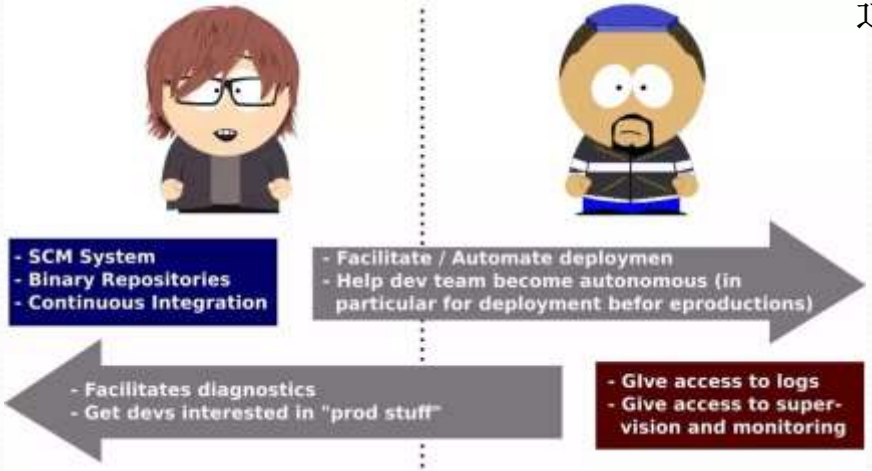
DevOps应运而生



软件开发流程



运维和用户



DevOps方式

持续集成

- 持续集成（Continuous Integration），缩写为CI，是一项软件开发实践。
- 在持续集成中，团队成员频繁集成他们的工作成果，一般每人每天至少集成一次，也可以多次。每次集成会经过自动构建（包括自动测试）的检验，以尽快发现集成错误。许多团队发现这种方法可以显著减少集成引起的问题，并可以加快团队合作软件开发的速度。
- 持续集成的目的，就是让产品可以快速迭代，同时还能保持高质量。它的核心措施是，代码集成到主干之前，必须通过自动化测试。只要有一个测试用例失败，就不能集成。
- 持续集成的两大优点：
  - 快速发现错误。每完成一点更新，就集成到主干，可以快速发现错误，定位错误也比较容易。
  - 防止分支大幅偏离主干。如果不是经常集成，主干又在不断更新，会导致以后集成的难度变大，甚至难以集成。

持续交付

- 持续交付（Continuous Delivery），缩写为CD，指的是频繁地将软件的新版本，交付给质量团队或者用户，以供评审。如果评审通过，代码就进入生产阶段。
- 持续交付可以看作持续集成的下一步。它强调的是，不管怎么更新，软件是随时随地可以交付的。
- 持续交付描述的是软件开发，是从原始需求识别到最终产品部署到生产环境这个过程中，需求以小批量形式在团队的各个角色间顺畅流动，能够以较短地周期完成需求的小粒度频繁交付。频繁交付周期带来了更迅速的对软件的反馈，并且在这个过程中，需求分析，产品的用户体验和交互设计、开发、测试、运维等角色密切协作，相比于传统的瀑布式软件团队，更少浪费。
- 持续交付的优势：
  - 快速发布
  - 缩短迭代周期
  - 软件发布标准化
  - 交付进度可视化
  - 减少浪费

持续部署

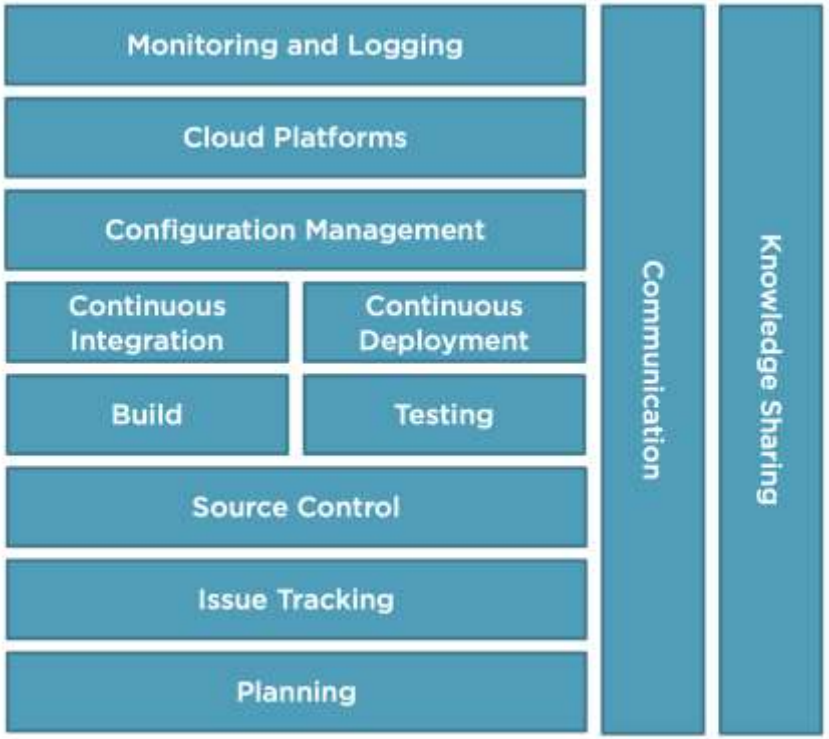
- 持续部署是指当交付的代码通过评审之后，自动部署到生产环境中。持续部署是持续交付的最高阶段，这意味着所有通过了一系列的自动化测试的改动都将自动部署到生产环境。
- 持续部署的工作流程：开发人员提交代码，持续集成服务器获取代码，执行单元测试，根据测试结果决定是否部署到预演环境，如果成功部署到预演环境，进行整体验收测试，如果测试通过，自动部署到产品环境中，全程自动化高效运转。
- 持续部署的优点：可以相对独立地部署新的功能，并能快速地收集真实用户的反馈。



成功的关键

- 文化冲击
  - 平稳的文化过渡是让DevOps获得长期成功应用和增强发布软件产品的综合能力的关键。第一步是，明确DevOps的定义，调动开发和运营部门之间的协作，鼓励运营人员采纳软件开发方法，并利用云计算基础设施来完成真实的测试和代码部署。
- 齐全工具箱
  - 想要超越文化的影响，组织还必须依靠各种 DevOps 工具。例如，开发人员编写代码需要工具、QA测试人员需要用工具完成新版软件的部署，环境准备、将新代码在测试系统和生产系统之间迁移也必须用到云资源调度工具。
  - 工具本身不是问题，重要的是能够让各种工具互相配合，在软件的生命周期内提供支持。

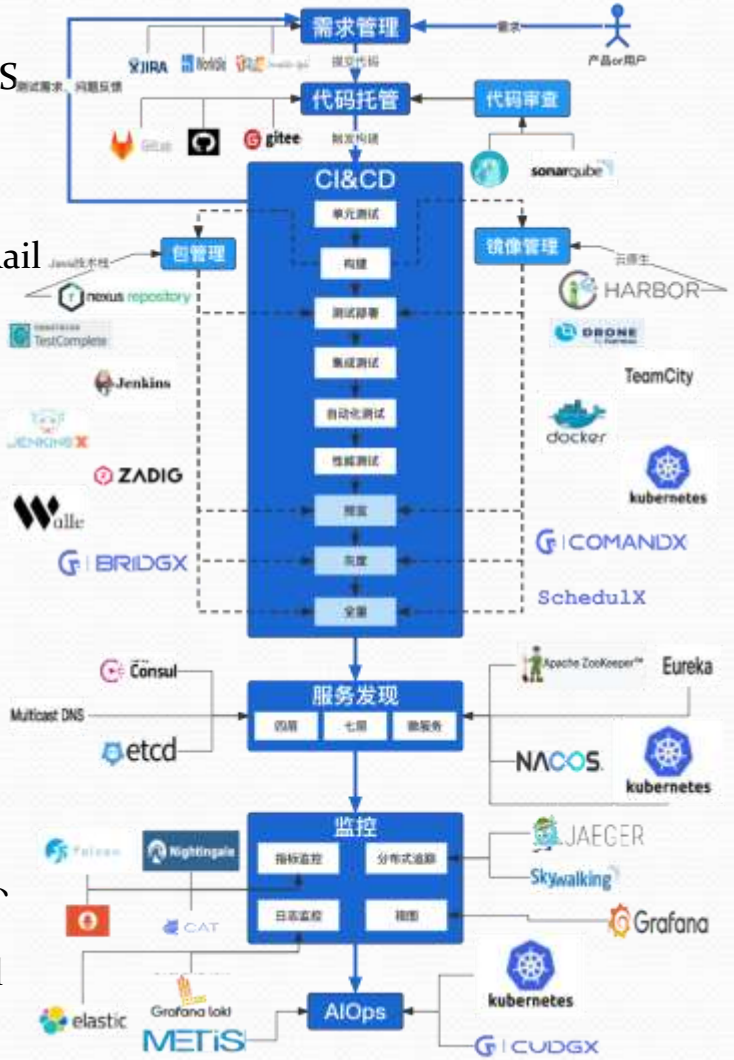
DevOps生态圈中工具链



- 编码：代码开发和审阅，版本控制工具、代码合并工具
- 构建：持续集成工具、构建状态统计工具
- 测试：通过测试和结果确定绩效的工具
- 打包：成品仓库、应用程序部署前暂存
- 发布：变更管理、发布审批、发布自动化
- 配置：基础架构配置和部署，基础架构即代码工具
- 监视：应用程序性能监视、最终用户体验
- 广泛使用的热门工具：Docker（容器化）、Jenkins（持续集成）、Puppet（基础架构构建）、Vagrant（虚拟化平台）

工具类型及对应的不完全列表

- 代码管理（SCM）：GitHub、GitLab、BitBucket、SubVersion、TFS
- 构建工具：Ant、Gradle、maven
- 自动部署：Capistrano、CodeDeploy
- 持续集成（CI）：Bamboo、Hudson、Jenkins
- 配置管理：Ansible、Chef、Puppet、SaltStack、ScriptRock GuardRail
- 容器：Docker、LXC、Rkt、第三方厂商如AWS
- 编排：Kubernetes、Apache Mesos、DC/OS
- 服务注册与发现：Zookeeper、etcd、Consul
- 脚本语言：python、ruby、shell
- 日志管理：ELK、Logentries
- 系统监控：Datadog、Graphite、Icinga、Nagios
- 性能监控：AppDynamics、New Relic、Splunk
- 压力测试：JMeter、Blaze Meter、loader.io
- 预警：PagerDuty、pingdom、厂商自带如AWS SNS
- HTTP加速器：Varnish
- 消息总线：ActiveMQ、SQS
- 应用服务器：Tomcat、JBoss
- Web服务器：Apache、Nginx、IIS
- 数据库：MySQL、Oracle、PostgreSQL等关系型数据库；cassandra、mongoDB、redis等NoSQL数据库
- 项目管理（PM）：Jira、Asana、Taiga、Trello、Basecamp、Pivotal Tracker

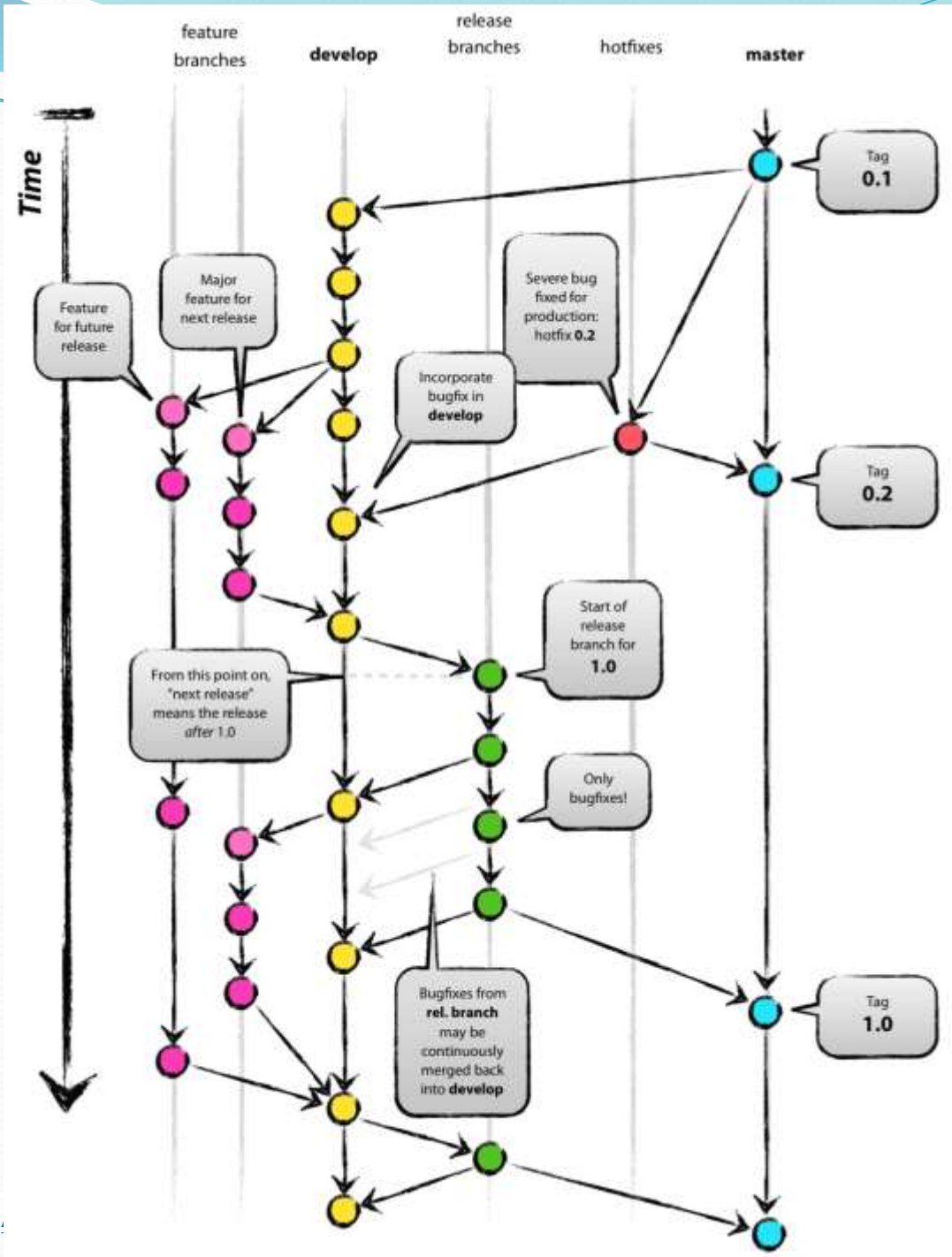


软件进化

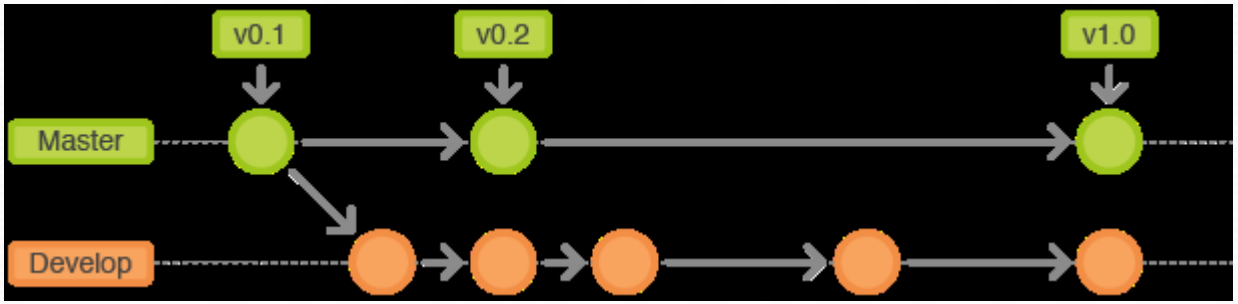
7 Gitflow工作流程

Git Flow

- Git Flow是一套基于git的工作流程，该工作流程围绕着project的发布(release)定义了一个严格的建立分支的模型。
- Git Flow的特点：
  - 特性驱动(Feature Driven)的建立分支法(Feature Branch Workflow)，即每一个特性(feature)的开发并不直接在主干上开发，而是在分支上开发，分支开发完毕后再合并到主干上。
- 这样做的优点是：
  - 还处于半成品状态的feature不会影响到主干
  - 各个开发人员之间做自己的分支，互不干扰
  - 主干永远处于可编译、可运行的状态

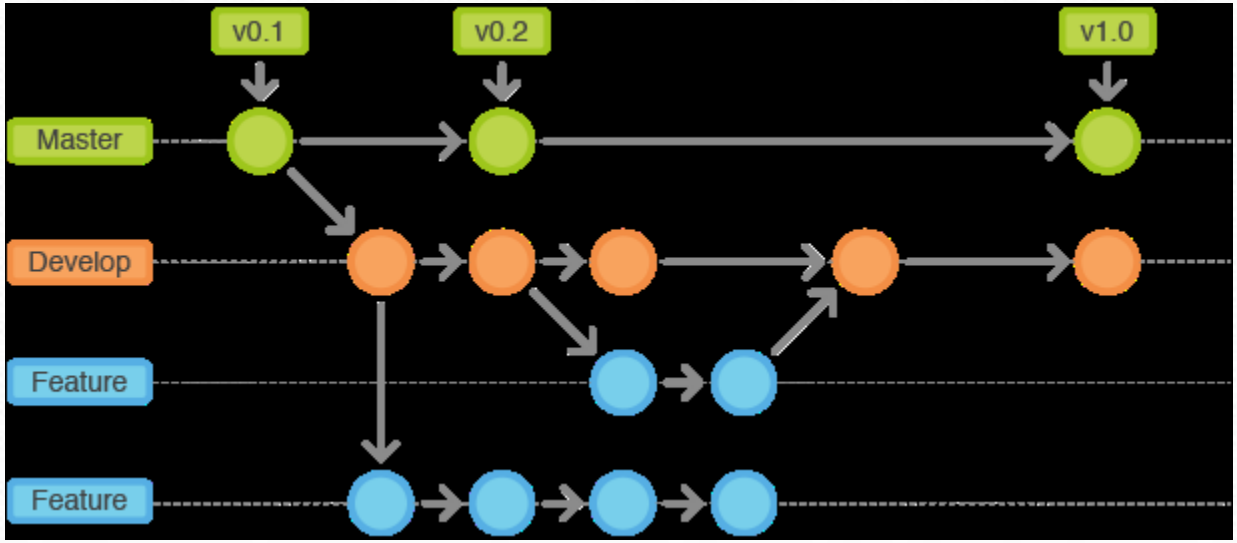


master和develop分支



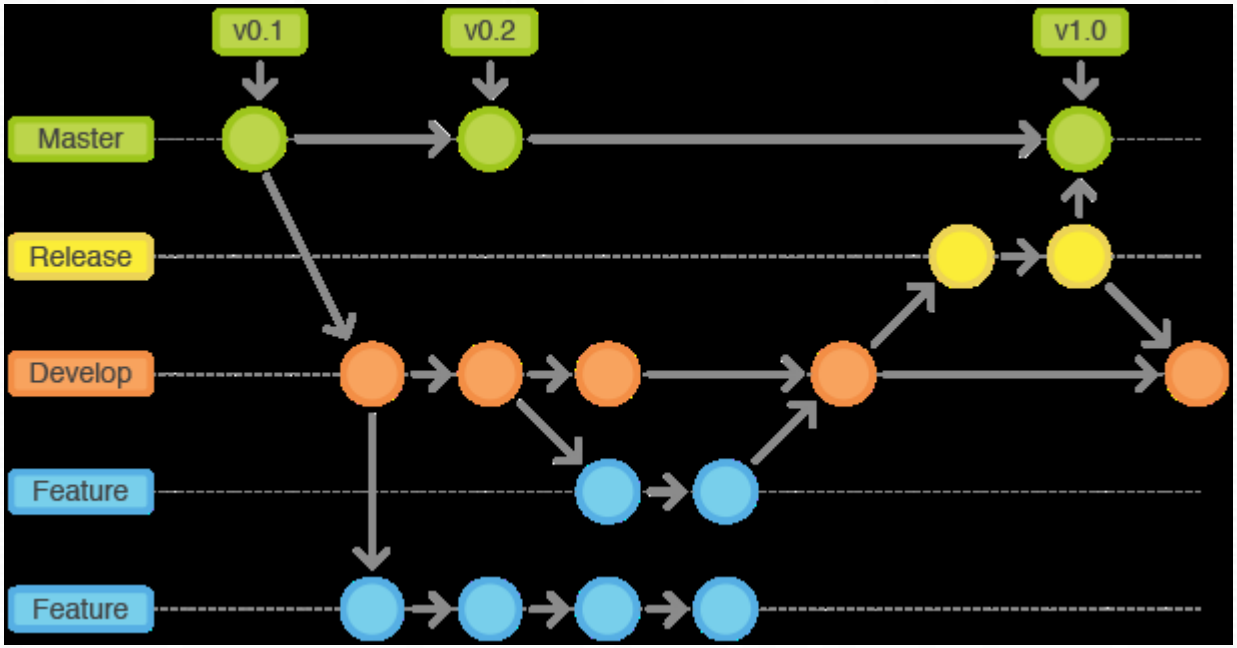
- master分支只存放历史发布(release)版本的源代码。各个版本通过tag来标记。上图里的v0.1和v0.2就是tag。
- develop分支则用来整合各个feature分支。开发中的版本的源代码存放在这里。

feature分支



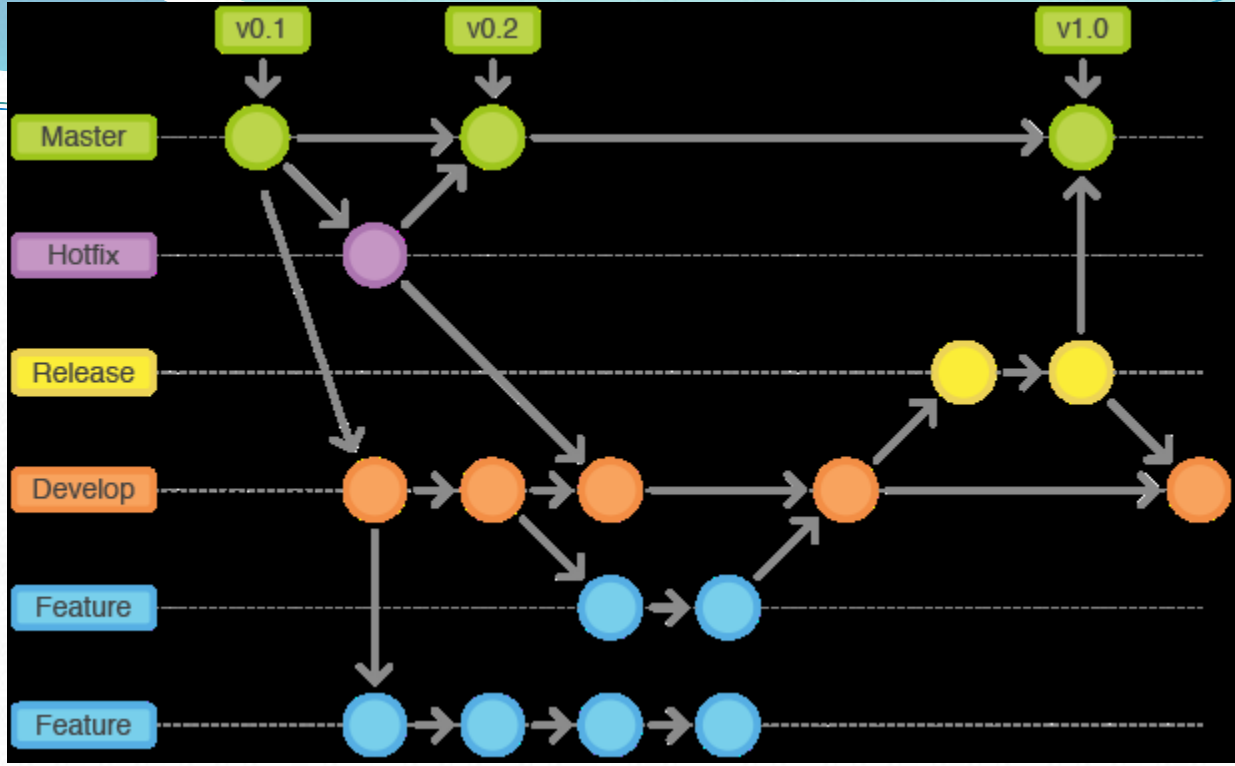
- 每一个特性(feature)都必须在自己的分支里开发，feature分支派生自develop分支。
- 当feature开发完毕后，要合并回develop分支。feature分支永远不会和master分支打交道。

release分支



- release分支不是正式发布产品的分支，是“待发布”分支，在这个分支干上做所有与发布有关的事情。但是，**绝对不会添加新的特性**。
- 当与发布相关的工作都完成后，release分支合并回develop和master分支。
- release分支的好处是，当一个团队在做发布相关的工作时，另一个团队则可以继续开发下一版本的东西。

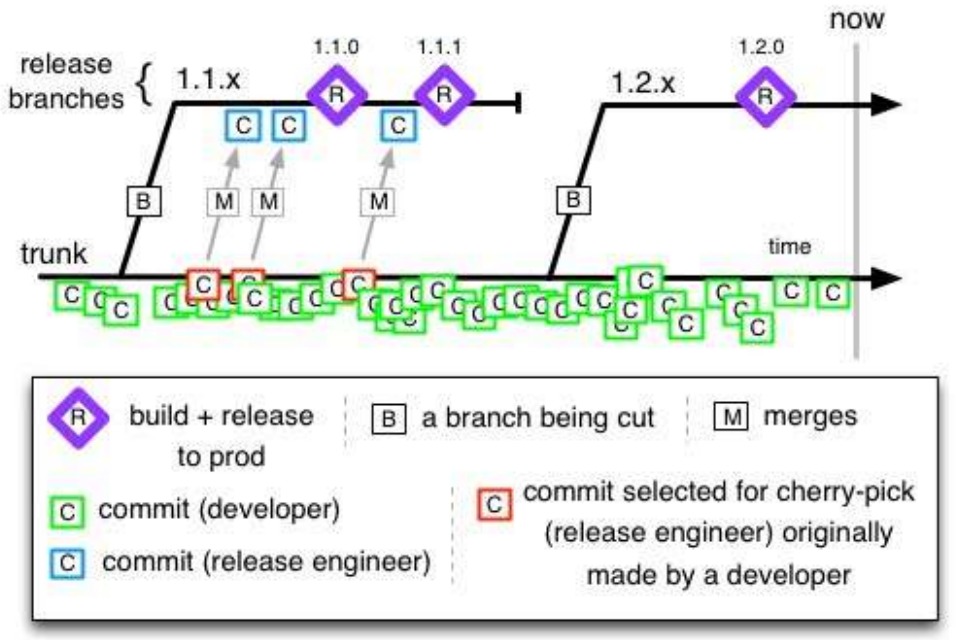
hotfix分支

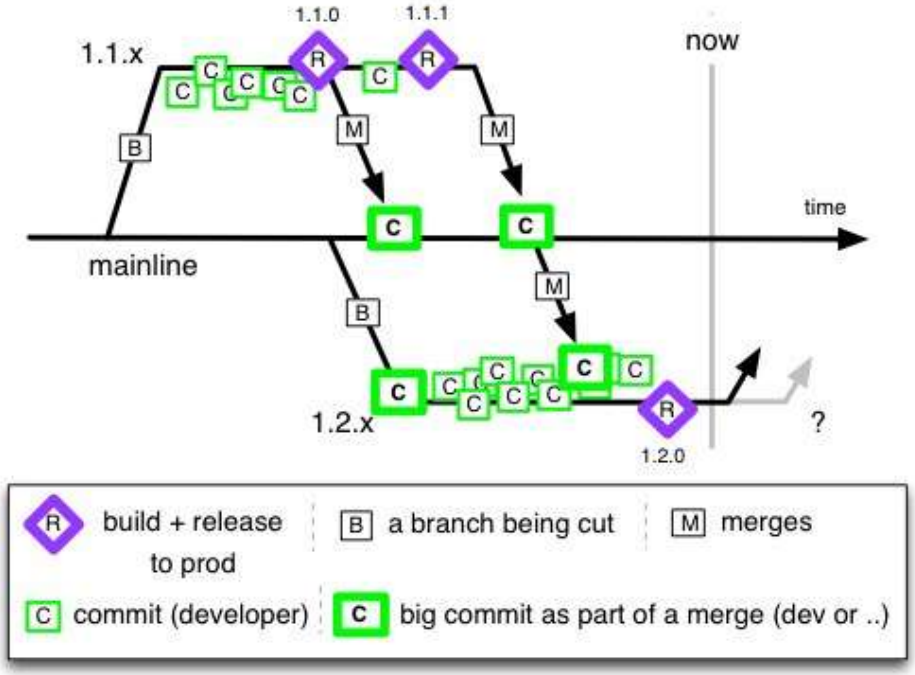


- 修复bug工作不适合在develop上做，这是因为develop分支上包含有未验证过的feature。
- 用户未必需要develop上的feature，但是，急需修复bug。这时需要新建hotfix分支。
- hotfix分支派生自master分支，仅仅用于修复bug，当bug修复完毕后，马上回归到master分支，然后发布一个新版本，比如v0.1.1。
- 同时hotfix也要合并回develop分支，这样develop分支就能获得bug修复的好处了。

Trunk Based Development

- Trunk Based Development，TBD，基于主干的开发。
- 关键点：
  - 同一个产品的所有开发人员共享一个Repository，有一个trunk，单一Developer或是Developer团队可以有自己的private branch，所有修改最后都会归并到主干
  - 只有在Release时才会有官方的分支，一般Developer不能对Release Branch作动作，只有Release Engineer可以更动Release Branch，当Release Branch完成它的任务，就会被砍掉
  - Bug先在trunk修好，之後把Commit合并到Release Branch，而不是在Release Branch修好再整合到trunk，這樣可以把修改Release Branch的人限制在最小程度。





课堂讨论：

(1) 什么是软件可维护性？讨论软件工程与可维护性关系。

(2) 机构的管理层要求你完成一个系统的评估，并且提出他们希望评估结果表示系统已经过时、应该被更换。这意味着许多系统维护人员将成为冗员。而你的实际评估表明该系统维护良好，属于高质量和高业务价值的系统。你如何向机构管理层报告评估结果？